Welcome to #database-system! v4.0

Post-Midterm starts Page 12
Old relevant stuff from #isdb starts at Page 42
Extras from Page 52; Blockchain from Page 58

What will we focus on this semester:

```
- Transaction
- Data Warehouse
- Distributed Database i.e. Blockchain
- Byzantine Consensus i.e. how will we synchronize updates without a centralized figure
.
.
and more
```

```
### Section 1: Transaction ###
```

- **What is transaction? (for DB)**
  -> a logical unit of work in which **integrity constraints** are allowed to be *violated*
- **Transaction recovery (what if transaction fails)?**
- **Transaction scheduling?**
- **Concurrency Control**

What is transaction? (for DB)
-> a logical unit of work in which **integrity constraints** are allowed to be *violated*

Transaction Concepts (Chapter 14)
Answer **at least 4 points** below in exam for full marks

```
- A transaction is a unit of program execution that accesses and possibly updates various
database items
- A transaction must see a consistent database
- During transaction execution, the database may be temporarily inconsistent
- When the transaction completes successfully (is committed), the database must be
consistent
- After a transaction commits, the changes it has made to the database persists, even if
there are system failures
- Multiple transaction can be executed in parallel
```

- **Two main issues:**
  -> **Failure of various kind** i.e. hardware failure and system crashes
  -> **Concurrent execution** of multiple transactions

What is the Integrity Rule/Constraint?
-> rules that **enforce correctness** of the logical data structure; see more in #isdb
-> in principle, rules should be declared by DB, enforced by DBMS and triggered by operation (before/after)

- **Integrity Preservation and Correctness** should be our main goal for transaction, not speed - else it's fcking useless
- **Sync Points** marks the *begin* and the *end* of transaction (this will be discuss in full detail tomorrow (edited)

ACID Properties
A transaction is a unit of program execution that accesses and possibly updates various data items. To preserve the integrity of data, the database must ensures:

Atomicity
Either all operations of the transaction are properly reflected in the database, or none at all

Consistency
Execution of a transaction in isolation preserves the data correctness of the database.

Isolation
Although multiple transaction may execute concurrently, each transaction must be unaware of other concurrently executing transactions. Intermediate transaction results must be hidden from other concurrently executed transactions.
That is, for every pair of transaction, T1 and T2, it appears to T1 that either T1 finishes first before T2 starts, or T2 finishes up first before T1

Durability
After a transaction completes successfully, the changes it has made to the database persist even if there are system failures

Please note that ACID properties are merely guidelines for transactions, and thus **totally unrelated** to Relational DB which are data model representations (edited)

ACID properties *are recommendation only and not mandatory*-> Thera are some of the exceptions:
- In case of system failure, in certain conditions we want the system to continue from its point of failure -> disregarding **Atomicity**
This way we only retry certain steps and not the whole process, also called "**Nested Transactions**"
This is use to handle long-duration transactions
- In distributed transactions, two-phase commit (local and global commit, such as in banks and git) -> disregarding **Consistency** in most cases
If data correctness and consistency is the main issue, then a centralized database is a *must*
If we can tolerate "Eventual Consistency" (i.e. Singapore DB update a day later than their US HQ), then Distributed Database can be use here
- In case where we need transaction to be arrange sequentially -> disregarding **Isolation**
Applies in "Concurrent Schedule" (where two execution overlaps) and "Serial Schedules" (A comes before B, ...)
This two can be combined into "Concurrent Serializable Schedule" which we wants rather than an actual Serial ones
- Changes are only write back from Commit/Rollback from the main memory to the disc when that file is no longer busy -> disregarding **Durability** (edited)

Level of Consistency in SQL-92 (Lower is faster but only approx. estimate rather than definite value)
- **Serialization** -> Default
- **Repeatable Read** -> Only committed records to be read, repeated reads of the same record *must return* the same value. However, a transaction may not be serializable - it may find some records inserted by a transaction but not others
- **Read committed** -> Only committed records can be read, but successive reads of record may return differently (but committed values)
- **Read uncommitted** -> even uncommitted records may be read

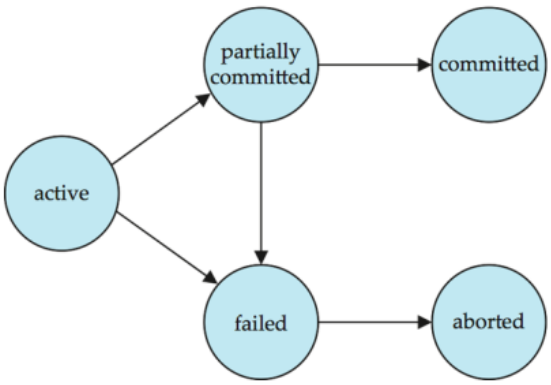
Best Practice:
- ***Do not*** perform a transaction in the "Interactive SQL mode" (as opposed to "Embedded SQL mode" where codes are embedded into the software) **since the system locks the DB** from allowing other people to update the DB

Sync Points
1) **Commit** Confirm Operation(s) logically from the previous sync point to the commit point
2) **Rollback** Cancel Operation(sS) logically from the previous sync point to the roll back point

Commit *is a promise* by the DBMS transaction manager **logically** (literally an "END TRX" for Nested Transactions, thus **itself a transaction**), not a force output update -> same goes for *Rollback* (edited)

Transaction State
- **Active** - the initial state; the transaction stays in this state while it is executing
- **Partially Committed** - after the final statement has been **executed**
- **Committed** - after successful completion (including DBMS's promise)
- **Failed** - after the discovery that normal transaction can no longer proceed
- **Aborted** - The transaction has been rollback to the state prior to the transaction, 2 ways to proceed after:
  - **Restart Transaction** if there's no internal logical error *or*
  - **Kill the Transaction**

Most DBMS use **locking** to lock the resources (read: *prevent file edit for everyone*) while transacting and will fully unlocked when committed -> **commit quickly is a must for performance** (edited)



Recovery System (Chapter 16)
- **(Individual) Transaction Failure** -> Failure of a particular transaction only; does not affect other transactions - these errors are handle automatically by DBMS without human intervention
  - **Logical Error**: transaction cannot complete due to internal error condition; System is still ok
  - **System Errors**: the database must terminate on active transaction due to error conditions (i.e. deadlocks)
- **System Crash** -> a power failure or other hardware or software failure causes the system to crash
  - **Fail-stop assumption**: non-volatile storage contents are assumed to not be corrupted by system crash; disc is assumed to be functional

Database system have numerous integrity checks to prevent corruption of disc data

- **Disk Failure**: a head crash or similar disk failure destroys all or part of disk storage

Destruction is assumed to be detectable: disk drives use ***checksums*** *to detect failure*

Storage Structure
- **Volatile Storage**: does not survive system crashes i.e. main memory, cache memory
- **Non-Volatile Storage**: survives system crashes i.e. disk, tape, flash memory, non-volatile (battery backed up) RAM
- **Stable Storage**: a mythical form of storage that ***survives all failures***; doable in practice by storing in multiple format of non-volatile storage; each with its own separate copies -> thus *expensive* to implement in practice (edited)

Data Access

- **Physical blocks** are those blocks residing on the disk.
- **Buffer Blocks** are the blocks residing temporarily in main memory.
- Block movements between disk and main memory are initiated through the following two operations:
  - **input**(B) transfers the physical block *B* to main memory.
  - **output**(B) transfers the buffer block *B* to the disk, and replaces the appropriate physical block there.

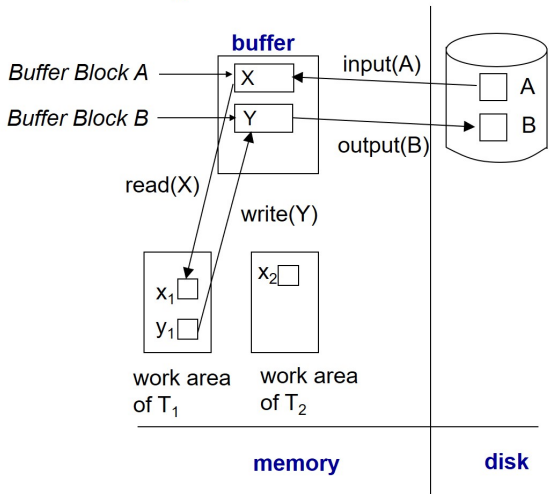
  DO NOT CONFUSE OUTPUT WITH COMMIT and can be done before/after commit

  -> output is physical while commit is logical
- Each transaction T*i* has its private work-area in which local copies of all data items accessed and updated by it are kept.

  T*i*'s local copy of a data item X is called x*i*.
- We assume, for simplicity, that **each data item fits in, and is stored inside, a single block**.

- Transaction **transfers data items between system buffer blocks and its private work-area** using the following operations:
  - **read**(X) assigns the value of data item X to the local variable x*i*.
  - **write**(X) assigns the value of local variable x*i* to data item {X} in the buffer block.
  - Both these commands may necessitate the issue of an input(BX) instruction *before the assignment*, if the block BX in which X resides is not already in memory.
- Transactions
  - Perform **read**(X) while accessing X for the first time;
  - All subsequent accesses are to the local copy.
  - After last access, transaction executes **write**(X).
- **output**(B*X*) need not immediately follow **write**(X). System can perform the **output** operation when it deems fit.

Example of Data Access



Recovery Control Problem Statements

1. Transaction **already commit** but the changes in the DBMS has **not yet been output** to the DB Space on the mass storage

   -> *What if the system failed, how could the committed transactions are recovered and the committed changes remains*
2. Transactions **are not yet committed** but changes in the DB buffer **have already been output** to the DB Space on the mass storage

   -> *What if the system failed, how could the partially outputted changes be cancelled out of the DB space*

The DBMS handle these problems automatically for us, but need a *proper setup*

Log-based Recover Classification
1) **Before Image Journal Only** -> Rollback Segmentation
- Most commonly used by DBMS provider as default; use only one log file
- It works (as in commit rollback works as usual)

Advantages:

- **No AIJ full problem** (since there's no AIJ) and thus less maintenance
- **Save storage space** (see above)
- During recovery - only undo and no redo -> **faster recovery**

Problems:

- Without AIJ, commit means moving DB buffer to DB space and thus **takes much longer time to commit** (8 minutes -> 19 hours for 13,000 row)
- Create **more traffic** (see above)

2) **After Image Journal Only** -> Redo Log File
- Commit works as usual (cttx, commit -> DB Space); popular with workgroup systems
- Without the old value, DB Buffer modification to DB space must **only be allowed after commit**

Advantages:

- **No BIJ full problem**
- **Save storage space**
- **Commit is a lot faster** (also called "Fast Commit" option) -> Good for small transaction i.e. Cashier Machines

Problems:

- **DB Buffer could be full before commit for large transactions** -> the server stops working (read: crash) (edited)

Log-based Recovery
- Follows **Write-Ahead Protocol** -> Write-Ahead Logging (WAL)
- Log records output to log file (stable storage) before corresponding DB buffer outputs to DB Space

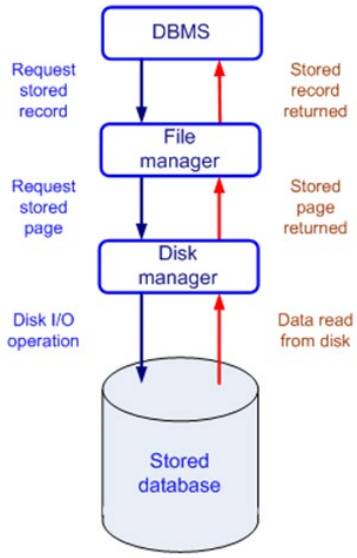
Checkpoints
- ***No need to redo all changes*** from the beginning of the entire log in case of system crashes
  1) Output all log records currently residing in main memory onto stable storage. (log record -> log file)
  2) Output all modified buffer blocks to the disk. (DB Buffer -> DB Space)
  3) Write a log record <**checkpoint**> onto stable storage. (<checkpoint> -> log file)
- Can be set up as frequently as we want, but there are mandatory checkpoints in places
- Good DBMS remove old the AIJ automatically to prevent Full AIJ error (cheaper options need DB Admin to do this manually)
- **Shrinking** -> the archival of log file(s) in AIJ to the archive to prevent full AIJ problem (edited)

Buffer Management
Ensures data consistency and imposes a minimal amount of overhead on interactions

Log Record Buffering
- **Log record buffering**: log records are buffered in main memory instead of being directly output to stable storage
  - Log records are output to stable storage when a block of log records in the **buffer is full** (or a log force operation is executed)
- **Log force** is performed to commit a transaction by forcing all its Log records (including the commit record) to stable storage.
- Several log records can be output using a single output operation -> **reducing the I/O cost**

Rules for Log Record Buffering

- Log records are output to stable storage **in the order in which they are created**
- Transaction T *i* **enters the commit state** only when the log record <**T *i* commit**> has been output to stable storage
- Before a block of data in main memory is output to the database, all log records pertaining to data in that block **must have been output to stable storage**
  - This rule is called the **Write-Ahead Logging** (WAL) rule; only requires undo information to be output. (edited)



Database Buffering

- **Database** maintains an in-memory buffer of data blocks
  - If buffer is full -> an existing block needs to be removed from buffer for the new block
  - If the block chosen for removal has been updated, it must be output to disk
- If a **block with uncommitted updates is output to disk**, log records with undo information for the updates are output to the log on stable storage first
  - *(Write-Ahead Logging)*
- **No updates should be in progress** on a block when it is output to disk; Can be ensured as follows.
  - **Before writing a data item**, transaction acquires exclusive lock on block containing the data item
  - **Lock can be released** once the write is completed.
    - Such locks held for short duration are called ***latches***.
  - **Before a block is output to disk**, the system acquires an exclusive latch on the block
    - **Ensures no update** can be in progress on the block (edited)

DB BackUp Concept

This is done in situation where you loses the DB Space but not the log files:

- **Volume BackUp**: BackUps everything
  - Can be done nearly instantly
  - Takes fckton of space
- **Incremental BackUp**: Back up only the changes
  - Takes much less space
  - Take hours to recreate the log
- **Cumulative (Incremental) BackUp**: Simply rollforward the changes in advance every morning/with log archive
  - Roll forward from previous log -> failover issue (bugs from system fail persists)
  - Very quick recovery time while is pretty lean on space (edited)

Database Export/Import

THIS IS NOT BACKUP OR RECOVERY

- Database Transportation
- Database Reorganization

Export: Data -> Command

Import: Command -> Data (edited)

Shadow Paging

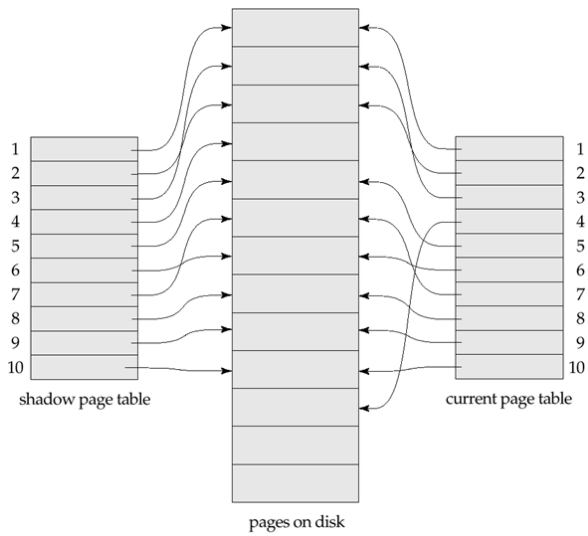
An alternative to log-based recovery; this scheme is useful if transactions execute serially

Idea: maintain two page tables during the lifetime of a transaction – the **current page table**, and the **shadow page table**

- Store the shadow page table in **nonvolatile storage**, such that state of the database prior to transaction execution may be recovered.
  - Shadow page table is never modified during execution
- To start with, both the page tables are identical. **Only** current page table is used for data item accesses during execution of the transaction.
- Whenever any page is about to be written for the first time
  - A copy of this page is made onto an unused page.
  - The current page table is then made to point to the copy
  - The **update** is performed on the copy
- **To commit** a transaction :
  1. **Flush all modified pages** in main memory to disk
  2. **Output current page** table to disk
  3. Make the current page table the new shadow page table, as follows:
     - Keep a pointer to the shadow page table at a fixed (known) location on disk.
     - To make the current page table the new shadow page table, update the pointer to point to current page table on disk

  Once pointer to shadow page table has been written, transaction is committed.
- **No recovery is needed** after a crash — new transactions can start right away, using the shadow page table.
- Pages not pointed to from current/shadow page table should be freed (garbage collected).

Example: Microsoft Word and other Office software



Advantages of SP

- No overhead of writing log records -> **instant recovery**
- Recovery is trivial

Disadvantages of SP

- Copying the entire page table is very expensive
  - Can be reduced by using a page table structured like a B+-tree
    - No need to copy entire tree, only need to copy paths in the tree that lead to updated leaf nodes
- Commit overhead is high even with above extension
  - Need to flush every updated page, and page table
- Data gets fragmented (related pages get separated on disk)
- After every transaction completion, the database pages containing old versions of modified data need to be garbage collected
- Hard to extend algorithm to allow transactions to run concurrently
  - Easier to extend log based schemes

Concurrent Executions

Multiple transactions are allowed to **run concurrently** in the system.

Advantages are:

- **Increased processor and disk utilization**, leading to better transaction throughput
  - E.g. one transaction can be using the CPU while another is reading from or writing to the disk
- **Reduced average response time for transactions**: short transactions need not wait behind long ones.

Concurrency control schemes

A mechanisms to **achieve isolation**

- To control the interaction among the concurrent transactions in order to **prevent them from destroying the consistency** of the database

Concurrent Schedule -> doing multiple transactions at the same time

Concurrent Serializable Schedule -> doing multiple transactions at the same time **while preserving consistency (we want this)**

Schedule

A sequences of instructions that **specify the chronological order** in which instructions of concurrent transactions are executed

- A schedule for a set of transactions **must consist of all instructions** of those transactions
- **Must preserve the order** in which the instructions appear in each individual transaction.

- A transaction that **successfully completes** its execution will have a **commit instructions as the last statement**
  - By default, transaction assumed to execute commit instruction as its last step
- A transaction that **fails** to successfully complete its execution will have an **abort instruction as the last statement** (edited)

Conflicting Instructions

Instructions I_i and I_j of transactions T_i and T_j respectively, conflict if and only if **there exists some item Q accessed by both I_i and I_j**, and **at least one of these instructions wrote Q**.

1. I_i = read(Q), I_j = read(Q). I_i and I_j don't conflict.
2. I_i = read(Q), I_j = write(Q). They conflict.
3. I_i = write(Q), I_j = read(Q). They conflict
4. I_i = write(Q), I_j = write(Q). They conflict

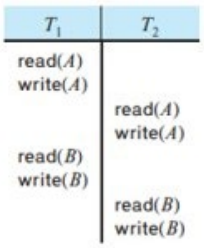


Figure 17.6 Schedule 3—showing only the read and write instructions.

Conflict Serializability

A schedule is **conflict serializable** if it is conflict equivalent to a serial schedule

- 17.6, 17.7, 17.8 are conflict equivalent schedules, each of them can be transformed into one another by the swaps of non-conflict instructions
- Since 17.8 is a serializable schedule, 17.6 and 17.7 are therefore conflict serializable schedules (see above)

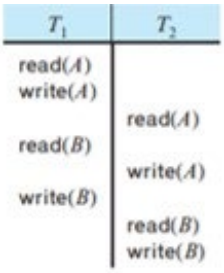


Figure 17.7 Schedule 5—schedule 3 after swapping of a pair of instructions.

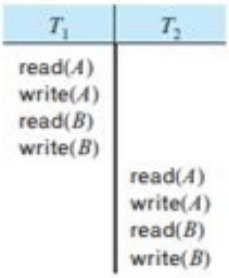


Figure 17.8 Schedule 6—a serial schedule that is equivalent to schedule 3.

View Serializability

- Let S and S´ be two schedules with the same set of transactions. S and S´ **are view equivalent** if the following three conditions are met, for each data item Q,

1. If in schedule S, transaction T*i* **reads the initial value of Q**, then in schedule S' also transaction T*i * **must read the initial value of Q**
2. If in schedule S transaction T*i* **executes read(Q)**, and that value was produced by transaction T*j * (if any), then in schedule S' also transaction T*i* **must read the value of Q** that was produced by the same write(Q) operation of transaction T*j* .
3. The transaction (if any) that **performs the final write(Q) operation** in schedule S must **also perform the final write(Q) operation** in schedule S'.

-> As can be seen, view equivalence is also **based purely on reads and writes alone**

- A schedule S is view serializable if it is **view equivalent to a serial schedule**
- **Every** conflict serializable schedule is also view serializable
- Below is a schedule which is view-serializable **but not conflict serializable**

T_{27}	T_{28}	T_{29}
read (Q)		
	write (Q)	
write (Q)		
		write (Q)

- What serial schedule is above equivalent to? -> a view serializable schedule
- Every view serializable schedule that is not conflict serializable **has blind writes** (T28 and T29; **write value without reading**; having no priorities)
  - view serializable is harder to check than conflict serializable (the latter is implemented in most DBMS)

Correct transaction means either conflict serializable or view serializable schedule -> having common based assumptions:

- Transactions operates with no failure
- All operation commits successfully (edited)

Recoverable Schedules

- Need for addressing the **effect of transaction failures** on concurrently running transactions
- **Recoverable Schedule**: if a transaction T*j* reads a data item previously written by a transaction T*i*, then the commit operation of T*i* appears before the commit operation of T*j*
- The following schedule (Schedule 11) **is not recoverable** if T*9* commits immediately after the read (edited)

T_8	T_9
read (A)	
write (A)	
	read (A)
	commit
read (B)	

- **If T*8* should abort, T*9*** would have read (and possibly shown to the user) an inconsistent database state. Hence, **database must ensure that schedules are recoverable** (edited)

Cascading Rollback

- A single transaction failure leads to **a series of transaction rollbacks.**
  Consider the following schedule where none of the transactions has yet committed (so the schedule is recoverable)
  If T*10* fails, T*11* and T*12* must also be rolled back -> **lead to the undoing of a significant amount of work**
- We don't want this -> **what we want is a Cascadeless Schedule** (edited)

T_{10}	T_{11}	T_{12}
read (A)		
read (B)		
write (A)		
	read (A)	
	write (A)	
		read (A)
abort		

Casecadeless Schedule

- Cascading rollbacks cannot occur; for each pair of transactions T*i* and T*j* such that T*j* reads a data item previously written by T*i*, the commit operation of T*i* appears before the read operation of T*j*.
- Every cascadeless schedule **is also recoverable**
- It is **desirable to restrict the schedules** to those that are cascadeless
- How to achieve recoverable and casecadeless? - > **Let the DBMS handle this** (see DBMS Isolation Level/Level of Consistency below) (edited)

Isolation Level/Level of Consistency in SQL-92

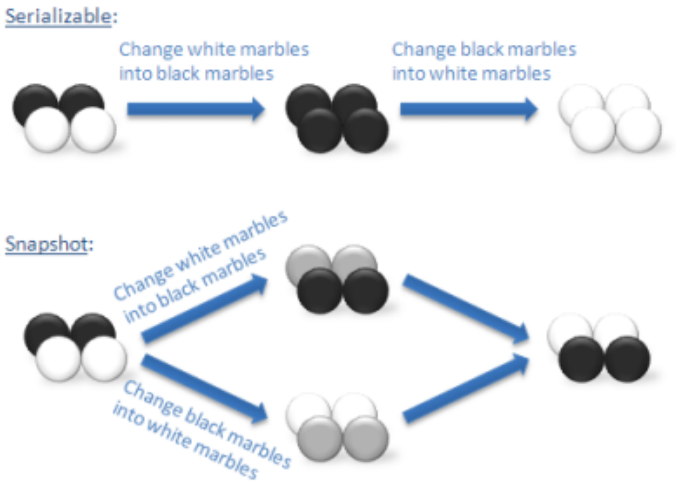
- **Serializable**: default (in theory) -> not the same as "serial"
  - Most DBMS is either view or conflict serializable schedule **in practice**
  - Also **Casecadeless** schedule (which means the source item is already committed)
  - Also **NO Phantom Phenomenon** (a new record are deleted/inserted from nowhere while reading)
  - i.e. no new account opened while summing the total money will be counted
- **Repeatable Read**: "Almost" the same as Serializable
  - only committed records to be read, repeated reads of same record **must return same value**
  - However, a transaction **may not be serializable**, it may find some records inserted by a transaction but not find others -> **Phantom Phenomenon may take place**
  - i.e. the amount from new bank accounts opened while sum is in progress may be counted
- **Read committed**: i.e. Cursor Stability algotithm, Snapshot Isolation -> many way to implement with different result; **dirty read**
  - Only committed records can be read -> **guarantees Cascadeless Schedule**
  - But **successive reads of record may return different** (but committed) values. -> **NOT View/Conflict serializable**
- **Read uncommitted**: Is often used to for "speed benchmark"
  - Even **uncommitted records may be read** -> Casecadeless Schedule are not guaranteed

- Can be overrule in the connection level and at the application program level
- Lower degree of isolation is faster for getting approx. result, but the results gets worst at each level (edited)
- In some DBMS the first two level are combined together with either name (see Oracle and DB2)
- **Some** database systems do not ensure serializable schedules by default

  E.g. Oracle and PostgreSQL by default support a level of consistency called **Snapshot Isolation** (not part of the SQL standard)

  - In **Serializable Isolation** level, **SQL Server acquires key** range locks and **holds them until the end of the transaction**
  - In **Snapshot Isolation** level, **SQL Server does not acquire any locks**

  It is possible for a concurrent transaction to modify data that a second transaction has already read, does not observe the changes and continues to read an old copy of the data.
- Locking is done by DBMS, don't worry about that (edited)



MIDTERM IS UP TO THIS POINT, CLOSE BOOK, NO CLASS NEXT WEEK

Final Exam (50%)

- What will be on Exam
  - Definition
  - Concepts
  - 1 Design Question for the Temporal Database (See ORM and 6NF)
  - Some calculation for Query Optimization
- Hard Copy is OK for Open Book (Written Down/Printed Out)
- Same style as last year (edited)

Welcome to #database-system! (Pt.2)

Concurrency Control Problems
1. **The Lost Update Problem** (2 concurrent transactions try to read and update the same data, thus pending changes ended up being overwritten)
2. **The Uncommitted Dependency Problem** (Data of another transaction in process is read before it commits)
3. **Inconsistent Analysis** (newly updated values is accounted for while another operation is in progress)
4. **Phantom Phenomenon** (newly inserted values is accounted for while another operation is in progress) -> We don't want this

SQL-92 Isolation Level and Concurrency Control
- **Serializable** -> Problem 1-4 are solved
- **Repeatable Read** -> Problem 1-3 are solved
- **Read Committed** -> Problem 2 is solved; **This is the default level set by vendors in Thailand**
  - **Snapshot Isolation** is one of the implementation for Read Committed, but does not belong directly on the SQL-92 Iso. Level
- **Read Uncommitted** -> Dirty Read could take place

By standard, **all isolation levels of DBMS still guarantees that Dirty Write (write on top of uncommitted data) will **not take place**
****Serializable** is not always the best one - this depends whether the overhead fits the work required or not (edited)

Concurrency Control
- Lock-Based Protocols
- Timestamp-Based Protocols
- Validation-Based Protocols
- Multiple Granularity
- Multiversion Schemes
- Insert and Delete Operations
- Concurrency in Index Structures

** Again, **We worry about the level of correctness, not the time** -> Concurrency Control rectify this issue for us
**This chapter is useful if we need to implement isolation level ourselves (edited)

Lock-Based Techniques
- A mechanism to **control concurrent access** to a data item
- 1. **Lock Primitives** (Lock-Unlock Instructions)
  Data items can be locked into various modes:
  - **Exclusive (X) mode**.
    - Data item can be both read as well as written.
    - X-lock is requested using **lock-X** instruction.
  - **Shared (S) mode**.
    - Data item can only be read.
    - S-lock is requested using **lock-S** instruction.
  - Commonly implemented using OS's **semaphore** (binary switch) -> Logical for OS, very Physical in the eyes of DBMS
  - However, only `Exclusive(X)` and `Shared(S)` are also known as **Lock Primitives**
  - This doesn't say anything about if others is currently locking the item
  - Only good for a **Point-Of-Time** correctness
  - **Lock requests** are made to concurrency-control manager (either DBMS or Application, but in this course it's the former). Transaction can proceed only after request is granted.

- 2. **Lock-Compatibility Matrix** -> think of *public toilet queues* , this alone doesn't solve all the 4 problems - sadly
    - What each status means:
        - `true` mean that we can just **proceed** to do our task directly.
        - `false` mean to just **wait until it is unlocked**, no transaction dies.
    - A transaction may be granted a lock on an item **if the requested lock is compatible with locks** already held on the item by other transactions.
        - To paraphrase, if the status is `true`
    - Any number of transactions **can hold shared locks on an item**, but if any transaction holds an exclusive on the item no other transaction may hold any lock on the item.
    - If a lock cannot be granted, the requesting transaction is made to **wait till all incompatible locks held by other transactions have been released**. The lock is then granted. (edited)

	S	X
S	true	false
X	false	false

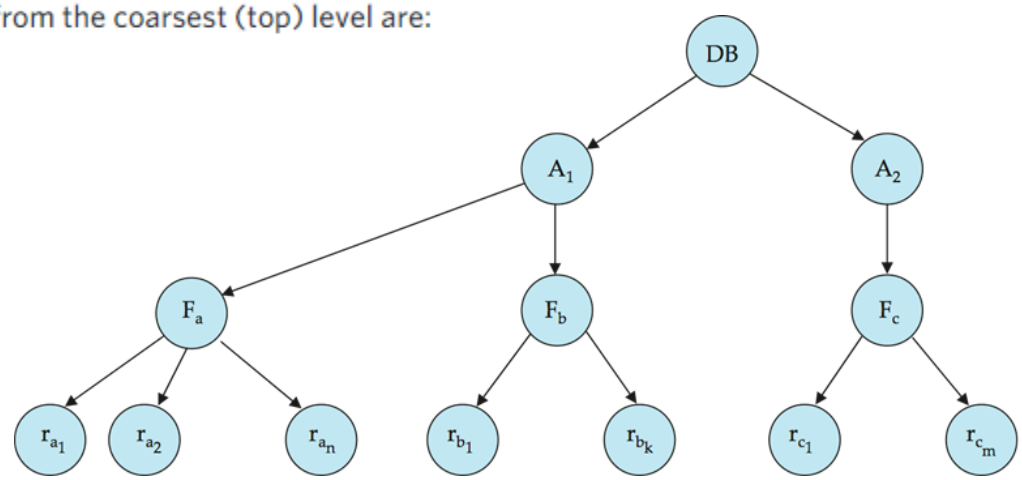
- 3. **Lock Protocol** (see below in detail)

Two-Phase Locking Protocol
- This is a protocol which ensures **conflict-serializable** schedules.
- All commercial DBMS use this according to Aj.Suphamit
- **Phase 1**: Growing Phase
    - transaction may obtain locks
    - transaction may not release locks
- **Phase 2**: Shrinking Phase
    - transaction may release locks
    - transaction may not obtain locks
- The protocol assures serializability. It can be proved that the **transactions can be serialized in the order of their lock points** (i.e., the point where a transaction acquired its final lock).
- This solves Problem 1, 3 - but **Uncommitted Dependency Problem** can still take place (same for 4: **Phantom Phenomenon**)
- Two-phase locking *does not* ensure freedom from **Deadlock Problem** (A wait for B, B wait for A) -> turns out this come from non-conflict serializable schedules
- Cascading Roll-back is *still* possible under two-phase locking.
    - **Strict Two-phase Locking** can avoid this. (a transaction must hold all its **exclusive locks** till it commits/aborts.)
        - Basically **X-Lock** at the beginning of concurrent transaction (this is called Wait Mode in certain DBMS products)
        - Adapt from Classical management techniques -> Paperwork are sorted first, and consolidated into one dude's job for each particular task
        - **X-Lock** can only be release at the sync point, **S-Lock** can be release at the start of the shrinking phase
        - This adds extra overhead for DBMS to check whether if it's X or S lock -> some DBMS just use Rigorous Two-Phase Locking straight away
    - **Rigorous Two-Phase Locking** is even stricter (**all locks** are held *till* commit/abort)
        - In this protocol transactions can be serialized in the order in which they commit.
        - Both **X-Lock** and **S-Lock** must wait for the shrinking phase before release
- To fix Problem 4(and 3), **a higher granularity is needed** (but *bigger granularity* means less concurrent operation but also requires less overhead) (edited)

Example of Granularity Hierarchy

The levels, starting from the coarsest (top) level are:

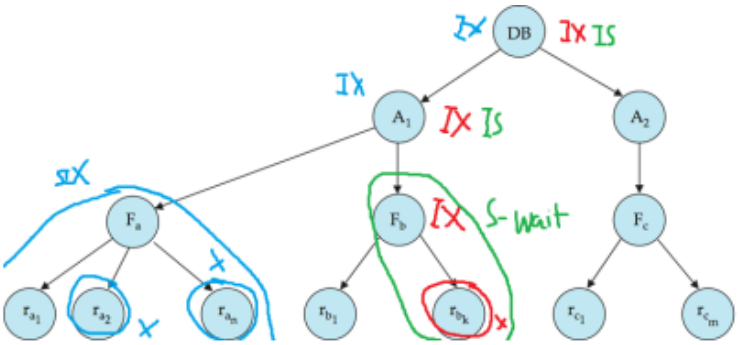
- **database**
- **area**
- **file**
- **record**



Intention Lock Modes

- ***In addition to* S** and **X lock modes**, there are three additional lock modes with multiple granularity:
  - **Intention-Shared (IS)**:
    - Indicates explicit locking at a lower level of the tree but **only with shared locks**.
  - **Intention-Exclusive (IX)**:
    - Indicates explicit locking at a lower level with **exclusive or shared locks**
  - **Shared and Intention-Exclusive (SIX)**:
    - The subtree rooted by that node is locked explicitly in shared mode and **explicit locking is being done at a lower level with exclusive-mode locks**.
    - Also called **Select for Update** mode
- Intention locks **allow a higher level node** to be locked in S or X mode without having to check all descendent nodes.
- Luckily, **all of this are now handle by DBMS**, a compatibility matrix for all modes are shown below: (edited)

	IS	IX	S	SIX	X
IS	true	true	true	true	false
IX	true	true	false	false	false
S	true	false	true	false	false
SIX	true	false	false	false	false
X	false	false	false	false	false



To solve all the 4 Problems, we need a **Strict Two-Phase Locking with Multiple Granularity using Intention Lock Mode**, or to paraphrase:

- A transaction must **hold all exclusive locks till it commits/aborts**, -> Strict Two-Phase Locking
- **Ability to scale up data size locking** (to fix Problem 4), -> Multiple Granularity
- Ability to do **explicit locking at lower level** when scale up data size -> Intention Lock Mode (edited)

Pessimistic Approaches
The way we consider these 4 problems for our concurrency planning

Optimistic Approaches exists i.e. Snapshot Isolation (allowing to read both the same set of data, assuming the same rows won't be updated - else the one commited first stays) -> a Read Committed (and not Conflict Serializable) (edited)

Weak Levels of Consistency
Is use so that the changes can be seen (near)-immediately, not so much about consistency

- **Degree-two consistency**:
  - a way to implement Read Committed (Serializable is Degree-four)
  - differs from two-phase locking in that **S-locks may be released at any time, and locks may be acquired at any time**
  - X-locks must be held till end of transaction
  - **Serializability is not guaranteed**, programmer must ensure that no erroneous database state will occur]
- **Cursor Stability**: -> Use for Stock Market updates
  - **For reads, each tuple is locked, read, and lock is immediately released** -> lock only the rows it's reading
  - X-locks are held till end of transaction
  - **Special case of degree-two consistency**
  - Can be guaranteed serializable by **setting the cursor to move forward only** by the programmer (edited)

Deadlock Prevention
- System is deadlocked *if* there is a set of transactions such that every transaction in the set is waiting for another transaction in the set.
- Deadlock prevention protocols ensure that the system will never enter into a deadlock state. Some prevention strategies:
  - Require that **each transaction locks all its data items before** it begins execution **(predeclaration)**.
  - Impose **partial ordering of all data items** and require that a transaction can lock data items only in the order specified by the partial order **(graph-based protocol)**.

- Following schemes use transaction timestamps for the sake of deadlock prevention alone.
- **wait-die scheme** — non-preemptive
  - Older transaction may **wait for younger one to release data item**.
  - Younger transactions **never wait for older ones**; they are rolled back instead. (always die)
  - **A transaction may die several times** before acquiring needed data item
- **wound-wait scheme** — preemptive
  - Older transaction **wounds (forces rollback) of younger transaction instead of waiting for it**.
  - **Younger transactions may wait for older ones.**
  - May be **fewer rollbacks** than wait-die scheme (edited)

Timestamp-Based Protocols

- **Each transaction is issued a timestamp** when it enters the system. If an old transaction T*i* has time-stamp TS(T*i*), a new transaction T*j* is assigned time-stamp TS(T*j*) such that TS(T*i*) <TS(T*j*).
- The protocol manages concurrent execution such that the time-stamps **determine the serializability order**.
- Only **guarantees conflict serializability** (but not *view serializability*)

- Timestamps can be implemented two ways:
  - **System Clock**: Easiest to implement, but may have issue in network-related systems if the clocks are not in-sync
  - **Logical Counter**: The timestamp is equal to the value of counter - useful in `Distributed Systems`, see **Lamport timestamp**

- In order to assure such behavior, the protocol maintains for each data Q two timestamp values:
  - **W-timestamp(Q)** is the **largest time-stamp** of any transaction that executed write(Q) successfully.
  - **R-timestamp(Q)** is the **largest time-stamp** of any transaction that executed read(Q) successfully.
- The **timestamp ordering protocol** ensures that any conflicting read and write operations are executed in timestamp order.

- Suppose a transaction T*i* issues a read(Q):
  - If **TS(T*i*)≤ W-timestamp(Q)**, then T*i* needs to read a value of Q that was already overwritten.
    Hence, the read operation **is rejected**, and T*i* is rolled back.
  - If **TS(T*i*)≥ W-timestamp(Q)**, then the read operation **is executed**, and R-timestamp(Q) is set to max(R-timestamp(Q), TS(T*i*)).
  - This concept is similar to the **Deadlock Prevention** listed above (edited)
- Suppose that transaction T*i* issues write(Q).
  1. **If TS(T*i*) < R-timestamp(Q)**, then the value of Q that T*i* is producing was needed previously, and the system assumed that that value would never be produced.
     Hence, the write operation is **rejected**, and T*i* is rolled back.
  2. **If TS(T*i*) < W-timestamp(Q)**, then T*i* is attempting to write an obsolete value of Q.
     Hence, this write operation **is rejected**, and T*i* is rolled back.
  3. **Otherwise**, the write operation **is executed**, and W-timestamp(Q) is set to TS(T*i*).

- **Timestamp Protocol use Rollback**, rather than Locking to prevent bad schedule -> ***fcking extremely slow*** (edited)

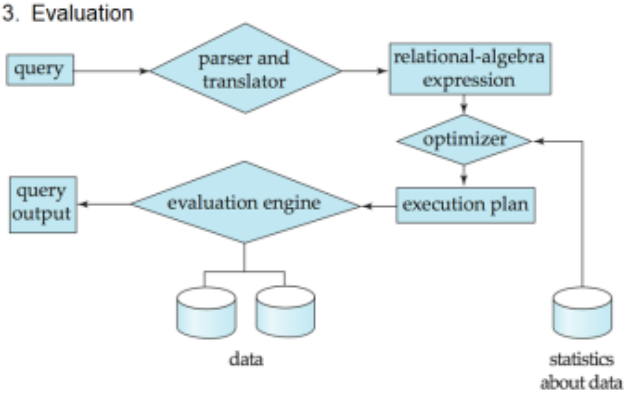
Multi-Version Scheme

- Multi-Version Scheme may works better than Multiple Granularity for Timestamp-based protocol:
  - This results in **Multi-version Two-Phase Locking**
    - **2PL** is used to solve ***problem no. 1 & 2***
    - **Multi-Version Scheme** is used to solve ***problem no. 3 & 4***
    - i.e. *Oracle (since ver. 7), PostgreSQL, MS SQL Server*
  - There's also **Multi-version Timestamp Ordering Protocol**:
    - **Timestamp Ordering Protocol** solves solve ***problem no. 1, 3 & 4***
    - **Multi-Version Scheme** is used to solve ***problem no. 3 & 4***
- Problem of Multi-Version is that ***this consumes fckton of memory*** (by the old days standard anyway)
  - **Undo Table Space** can be use to mitigate this issue
    - If r*O* is not found even in the undo table space? -> **r*O* will be rolled back** (edited)

Querry Processing
- The performance should be optimized in "How" part -> This should be handle by DBMS, rather than programmer for better productivity
- In the "Relational" side of things - programmer should only need to do the "What" part -> Relational Algebra is a basis for this (edited)

Basic Steps in Query Processing

1. Parsing and translation
2. Optimization
3. Evaluation



Relational Algebra ### (see more in CH2.pptx of `RDBMS5Special`)
- Procedural language
- Six basic operators
  - Select: **σ**
  - Project: **π**
  - Join: ⋈
  - Union: **∪**
  - Set Difference: **−**
  - Cartesian Product: **x**
  - Rename: **ρ**
- The operators take one or two relations as inputs and produce a new relation as a result. (edited)

No notes on Relational Algebra, jot down here

Basic Steps in Query Processing

- **Parsing and Translation**
  - Translate the query into its internal form. This is then translated into relational algebra.
  - Parser checks syntax, verifies relations
- **Evaluation**
  - The query-execution engine takes a query-evaluation plan, executes that plan, and returns the answers to the query. (edited)

Optimization

- A relational algebra expression **may have many equivalent expressions**
  - E.g., **σsalary<75000(πsalary(instructor))** is equivalent to **πsalary(σsalary<75000(instructor))** -> we do from inside-out
- Each relational algebra operation **can be evaluated using one of several different algorithms**
  - Correspondingly, a relational-algebra expression can be evaluated in many ways.
- **Annotated expression** specifying detailed evaluation strategy is called an **Evaluation-Plan**.
  - E.g., can use an index on salary to **find** instructors with `salary < 75000`,
  - or can perform complete **relation scan and discard** instructors with `salary ≥ 75000`
- **Query Optimization**: Amongst all equivalent evaluation plans choose the one with lowest cost.
  - Cost is estimated using statistical information from the database catalog
    - e.g. number of tuples in each relation, size of tuples, etc.
- In this chapter we study:
  - **How** to measure query costs
  - **Algorithms** for evaluating relational algebra operations
  - **How to combine algorithms** for individual operations in order to evaluate a complete expression (edited)

Analytics DB/Warehouse usually **keep data by column** (rather than the transaction-based ones which keep data by row) i.e. SyBase -> this needs a specialized DB

```
    PROS:
        - "Data Storage":
            Like all database systems, wide columnar databases enable users to store
data.
            They use a durable data storage system to prevent data loss and ensure
durability.
            Business users can use them to store their customers' details such as
names, email addresses, gender, and age.
        - "Data Manipulation and Sorting":
            Database users can sort and manipulate data directly from a columnar
database and there is no need to rely on the application.
        - "Self-Indexing":
            Some columnar databases can index columns to ensure effective data
processing, querying, and analysis.
        - "Scalability":
            Columnar databases are highly scalable because they store data in columns
rather than rows.
            The columns are easy to scale so they can store large volumes of data.
            This feature also allows users to spread data across many computing nodes
and data stores.
        - "Compression":
            Columnar databases are highly compressed compared to conventional
relational databases that store data by row.
            They allow users to optimize storage size.
    CONS:
        - "Unsuitable for all Transaction-related stuff"
            - Transactions are to be avoided or just not supported.
            - Queries with table joins can reduce high performance.
            - Record updates and deletes reduce storage efficiency.
        - "Hard to implement Partition"
            Effective partitioning/indexing schemes can be difficult to design.
```


- JSON actually also keep shit by column lmao (not the actual prof note) (edited)

- Single Table Cluster -> keep related things together after joining
- Multi-Table Cluster -> similar to above, but better; available in top brand name DBMS
- Most DBMS nowadays, if not all now automatically create an index on S number column
    - **Fast Join** is when we join two table using the same primary key columns (which are incidentally, index columns) (edited)

> There are 2 main query optimization technologies:
> - ***Rule-based Optimizer***: decides by using **fixed set of rules**; mostly depends on the way SQL statements are written (default, old method - last one was Oracle 6 in 1992)
> - ***Cost-based Optimizer***: decides by using **database statistics**; need to be pre-collected (better, modern ones)
> - ***Semantic Query Optimizer*** = Cost-based Optimizer + **Business Rules** (embedded in DBMS) for Query Optimization`

This chapter is entirely **Cost-based**
- The DBMS will revert back to **Rule-Based** if there's no statistics available (bad AF)
- See more in #isdb

- We can create **our own hints/plan to overrule the optimizer**
    - This is not necessary though - only useful for some very specific cases but otherwise **don't fcking do this** (edited)

No notes on Domain Relational Calculus (DRC) vs Interactive QVE, jot down here

Measures of Querry Cost

- For simplicity we just use the number of block transfers from disk and the number of seeks as the cost measures
    - tT – time to transfer one block
    - tS – time for one seek
- Cost for b block transfers plus S seeks
    b * tT + S * tS
- We ignore CPU costs for simplicity
- Real systems do take CPU cost into account
- We do not include cost to writing output to disk in our cost formulae
- Several algorithms can reduce disk IO by using extra buffer space
    - Amount of real memory available to buffer depends on other concurrent queries and OS processes, known only during execution
        - We often use worst case estimates, assuming only the minimum amount of memory needed for the operation is available
- Required data may be buffer resident already, avoiding disk I/O
    - But hard to take into account for cost estimation

Indexing and Hashing

Basic Concept

- Indexing mechanisms used to speed up access to desired data.
    - E.g., author catalog in library
- **Search Key** - attribute to set of attributes used to look up records in a file.
- An **index file** consists of records (called index entries) of the form

```
_____________________
| Search-Key || Pointer |
_____________________
```

- Index files are typically much smaller than the original file
- Two basic kinds of indices:
    - **Ordered indices**: search keys are stored in sorted order
    - **Hash indices**: search keys are distributed uniformly across "buckets" using a "hash function".

Index Evaluation Metrics

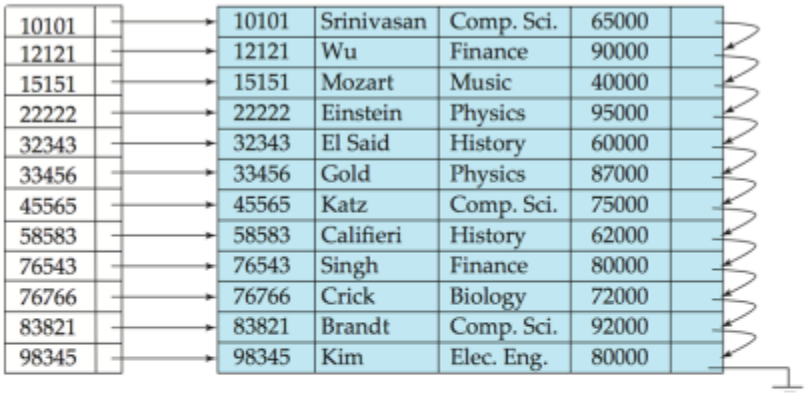
- Access types supported efficiently. E.g.,
    - records with a specified value in the attribute
    - or records with an attribute value falling in a specified range of values.
- Access time
- Insertion time
- Deletion time
- Space overhead (edited)

*Note: before we continue:

- **Index** is **access mechanism**
- **Key** is an **identifier**

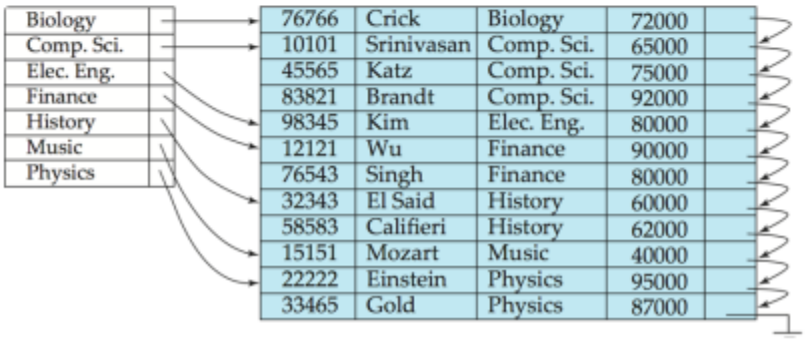
Dense Index Files

- **Dense index — Index record appears for every search-key value** in the file.
  - E.g. index on ID attribute of instructor relation



10101	Srinivasan	Comp. Sci.	65000	
12121	Wu	Finance	90000	
15151	Mozart	Music	40000	
22222	Einstein	Physics	95000	
32343	El Said	History	60000	
33456	Gold	Physics	87000	
45565	Katz	Comp. Sci.	75000	
58583	Califieri	History	62000	
76543	Singh	Finance	80000	
76766	Crick	Biology	72000	
83821	Brandt	Comp. Sci.	92000	
98345	Kim	Elec. Eng.	80000	

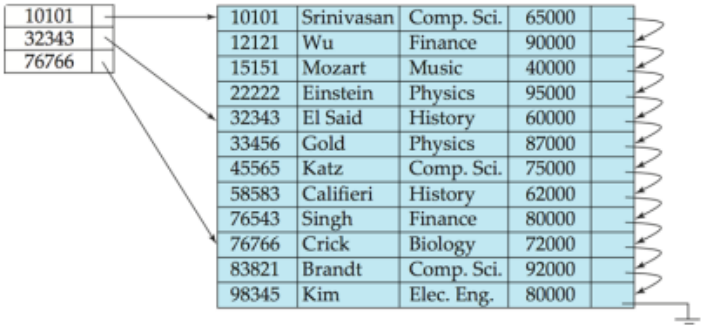
- Dense index on *dept_name*, with instructor file sorted on *dept_name*



Biology	76766	Crick	Biology	72000
Comp. Sci.	10101	Srinivasan	Comp. Sci.	65000
Elec. Eng.	45565	Katz	Comp. Sci.	75000
Finance	83821	Brandt	Comp. Sci.	92000
History	98345	Kim	Elec. Eng.	80000
Music	12121	Wu	Finance	90000
Physics	76543	Singh	Finance	80000
	32343	El Said	History	60000
	58583	Califieri	History	62000
	15151	Mozart	Music	40000
	22222	Einstein	Physics	95000
	33465	Gold	Physics	87000

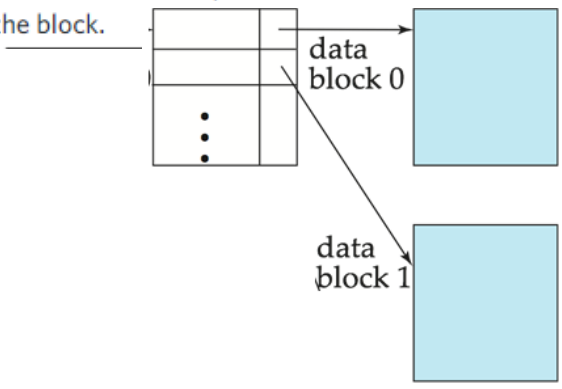
Sparse Index Key

- **Sparse Index**: contains index records for **only some search-key values**.
  - Applicable when records are sequentially ordered on search-key
- To locate a record with search-key value K we:
  - Find index record with largest search-key value < K
  - Search file sequentially starting at the record to which the index record points



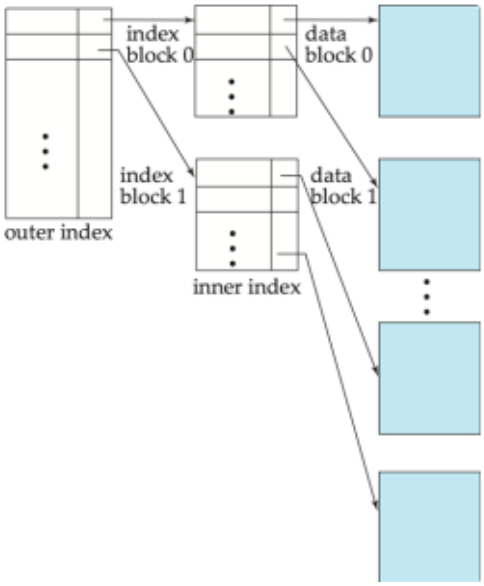
10101	10101	Srinivasan	Comp. Sci.	65000
32343	12121	Wu	Finance	90000
76766	15151	Mozart	Music	40000
	22222	Einstein	Physics	95000
	32343	El Said	History	60000
	33456	Gold	Physics	87000
	45565	Katz	Comp. Sci.	75000
	58583	Califieri	History	62000
	76543	Singh	Finance	80000
	76766	Crick	Biology	72000
	83821	Brandt	Comp. Sci.	92000
	98345	Kim	Elec. Eng.	80000

- Compared to dense indices:
  - **Less space and less maintenance overhead** for ***insertions*** and ***deletions***.
  - Generally **slower than dense index** for ***locating records***.
- **Good tradeoff**: sparse index with an index entry for every block in file, **corresponding to least search-key value** in the block.



Multilevel Index

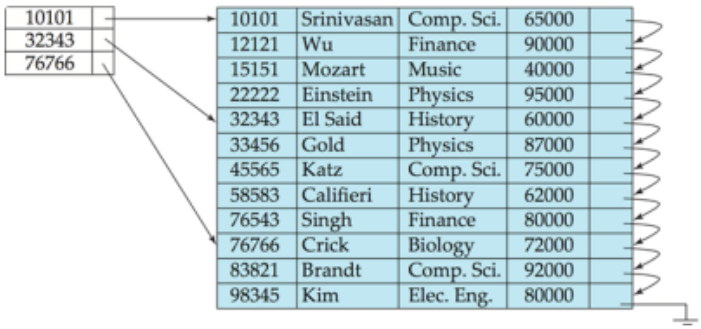
- If primary index does not fit in memory, access becomes expensive.
- Solution: treat primary index kept on disk as a sequential file and construct a sparse index on it.
  - **outer index** – a sparse index of primary index
  - **inner index** – the primary index file
- If even outer index is too large to fit in main memory, yet another level of index can be created, and so on.
- **Indices at all levels must be updated on insertion or deletion** from the file. (edited)



Index Update: Deletion

- **If deleted record was the only record** in the file **with its particular search-key** value, the **search-key is deleted from the index also.**
- **Single-level index entry deletion:**
  - **Dense indices**: deletion of search-key is **similar to file record deletion.**
  - **Sparse indices**:
    - if an **entry for the search key exists in the index**, it is deleted by **replacing the entry in the index with the next search-key value** in the file (in search-key order).
    - If the **next search-key value already has an index entry**, the **entry is deleted** instead of being replaced.

(edited)



Index Update: Insertion

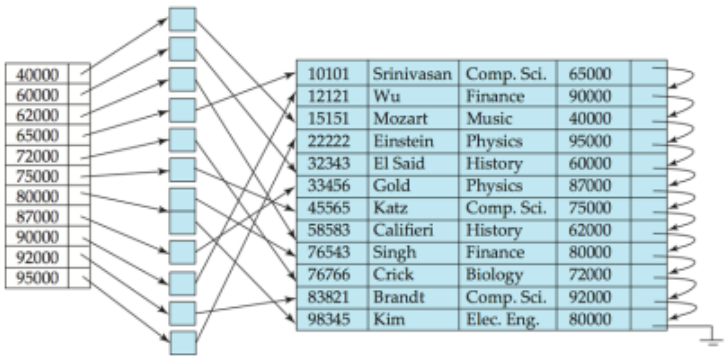
- **Single-level index insertion**:
  - Perform a lookup using the search-key value appearing in the record to be inserted.
  - **Dense indices** – if the search-key value does not appear in the index, insert it.
  - **Sparse indices** – if index stores an entry for each block of the file, no change needs to be made to the index unless a new block is created.
    - If a new block is created, the first search-key value appearing in the new block is inserted into the index.
- **Multilevel insertion and deletion**: algorithms are **simple extensions of the single-level algorithms**

Secondary Indices

- Frequently, one **wants to find all the records whose values in a certain field** (which is not the search-key of the primary index) satisfy some condition.
  - Example 1: In the instructor relation stored sequentially by ID, we may want to find all instructors in a particular department
  - Example 2: as above, but where we want to find all instructors with a specified salary or with salary in a specified range of values
- We can have a **secondary index with an index record for each search-key value**
- Index record points to a bucket that contains pointers to all the actual records with that particular search-key value.
- Secondary indices have to be **dense**

> 95% of the indexes in usage nowadays are secondary, not primary

(edited)



Primary and Secondary Indices

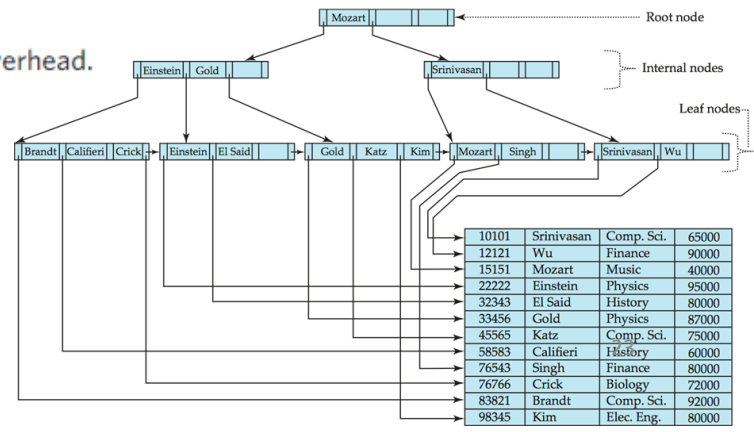
- Indices offer **substantial benefits when searching for records**.
- BUT: **Updating indices imposes overhead on database modification** --when a file is modified, every index on the file must be updated,
- **Sequential scan using primary index is efficient**, but a sequential scan **using a secondary index is expensive**
  - Each record access may fetch a new block from disk
  - Block fetch requires about 5 to 10 milliseconds, versus about 100 nanoseconds for memory access (edited)

B+ Tree Index Files

- B+ tree indices are **an alternative to indexed-sequential files**.
- **Disadvantage of indexed-sequential files**
  - **Performance degrades as file grows**, since many overflow blocks get created.
  - **Periodic reorganization** of entire file is **required**.

- **Advantage of B+ tree index files**:
  - **Automatically reorganizes itself** with small, local, changes, in the face of insertions and deletions.
  - **Reorganization of entire file is not required** to maintain performance.
- **(Minor) disadvantage of B+ trees**:
  - **extra insertion and deletion overhead**, space overhead.

- Advantages of B+ trees outweigh disadvantages
- **B+ trees are used extensively** (edited)



- A B+-tree is a rooted tree satisfying the following properties:
    - **All paths from root to leaf are of the same length**
    - Each **node that is not a root** or a leaf has between **⌈n/2⌉** and **n children.**
    - A **leaf node** has between **⌈(n–1)/2⌉** and **n–1 values**
    - Special cases:
        - If the root is **not a leaf**, it has **at least 2 children.**
        - If the root is **a leaf** (that is, there are no other nodes in the tree), it can have **between 0 and (n–1) values.**

Selection Operation

- **File scan**
- Algorithm **A1** (**linear search**). Scan each file block and test all records to see whether they satisfy the selection condition.
    - Cost estimate = br block transfers + 1 seek
        br denotes number of blocks containing records from relation r
    - If selection is on a key attribute, can stop on finding record
        cost = (br /2) block transfers + 1 seek
    - Linear search can be applied regardless of
        - selection condition or
        - ordering of records in the file, or
        - availability of indices
- Note: binary search generally does not make sense since data is not stored consecutively
    - except when there is an index available,
    - and binary search requires more seeks than index search

Selection Using Indices

- Index scan – search algorithms that use an index
    - selection condition must be on search-key of index.
- **A2** (***primary index, equality on key***). Retrieve a single record that satisfies the corresponding equality condition
    - Cost = (hi + 1) * (tT + tS)
- **A3** (***primary index, equality on nonkey***) Retrieve multiple records.
    - Records will be on consecutive blocks
        - Let b = number of blocks containing matching records
    - Cost = hi * (tT + tS) + tS + tT * b
- **A4** (***secondary index, equality on nonkey***).
    - Retrieve a single record if the search-key is a candidate key
        - Cost = (hi + 1) * (tT + tS)
    - Retrieve multiple records if search-key is not a candidate key
        - each of n matching records may be on a different block
        - Cost = (hi + n) * (tT + tS)
            - Can be very expensive!

Selections Involving Comparisons

- Can implement selections of the form $\sigma_{A \le V}(r)$ or $\sigma_{A \ge V}(r)$ by using
    - a linear file scan,
    - or by using indices in the following ways:
- **A5** (***primary index, comparison***). (Relation is sorted on A)
    - For $\sigma_{A \ge V}(r)$ use index to find first tuple ≥ v and scan relation sequentially from there
    - For $\sigma_{A \le V}(r)$ just scan relation sequentially till first tuple > v; do not use index
- **A6** (***secondary index, comparison***).
    - For $\sigma_{A \ge V}(r)$ use index to find first index entry ≥ v and scan index sequentially from there, to find pointers to records.
    - For $\sigma_{A \le V}(r)$ just scan leaf pages of index finding pointers to records, till first entry > v
    - In either case, retrieve records that are pointed to
        - requires an I/O for each record
        - **Linear file scan may be cheaper**

Algorithms for Complex Selections

- Disjunction: σθ1∨ θ2∨ . . . θn(r).
- **A10 (disjunctive selection by union of identifiers)**.
  - Applicable if all conditions have available indices.
    - Otherwise use linear scan.
  - Use corresponding index for each condition, and take union of all the obtained sets of record pointers.
  - Then fetch records from file
- **Negation**: σ¬θ(r)
  - Use linear scan on file
  - If very few records satisfy ¬θ, and an index is applicable to θ
  - Find satisfying records using index and fetch from file (edited)

Sorting

- We may build an index on the relation, and then use the index to read the relation in sorted order.
  May lead to one disk block access for each tuple.
- For relations that fit in memory, techniques like quicksort can be used.
  For relations that don't fit in memory, **external sort-merge** is a good choice.

External Sort-Merge

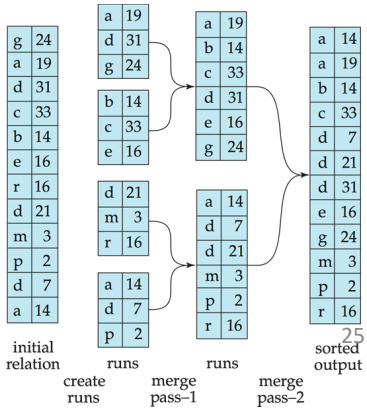
1. **Create sorted *runs***. Let i be 0 initially.
   - Repeatedly do the following till the end of the relation:
     (a) Read M blocks of relation into memory
     (b) Sort the in-memory blocks
     (c) Write sorted data to run Ri; increment i.
   Let the final value of i be N
2. **Merge the runs (N-way merge)**. We assume (for now) that N < M.
   1. Use N blocks of memory to buffer input runs, and 1 block to buffer output. Read the first block of each run into its buffer page
   2. repeat
      1. Select the first record (in sort order) among all buffer pages
      2. Write the record to the output buffer. If the output buffer is full write it to disk.
      3. Delete the record from its input buffer page.

```
if the buffer page becomes empty then
        read the next block (if any) of the run into the buffer.
```

   3. until all input buffer pages are empty

- If N≥M, several merge passes are required.
  - In each pass, contiguous groups of M - 1 runs are merged.
  - A pass reduces the number of runs by a factor of M -1, and creates runs longer by the same factor.
    - E.g. If M=11, and there are 90 runs, one pass reduces the number of runs to 9, each 10 times the size of the initial runs
  - Repeated passes are performed till all runs have been merged into one

Example: External Sorting using Sort-Merge



- **Cost analysis**:
  - Total number of merge passes required: ⌈logM–1(br/M)⌉.
  - Block transfers for initial run creation as well as in each pass is 2br
    - for final pass, we don't count write cost
      - we ignore final write cost for all operations since the output of an operation may be sent to the parent operation without being written to disk
    - Thus total number of block transfers for external sorting:

```
br ( 2 ⌈logM–1(br / M)⌉ + 1)
```

- **Cost of seeks**
  - During run generation: one seek to read each run and one seek to write each run
    - 2 ⌈br / M⌉
  - During the merge phase
    - Buffer size: bb (read/write bb blocks at a time)
    - Need 2 ⌈br / bb⌉ seeks for each merge pass
      - except the final one which does not require a write
    - Total number of seeks:

```
2 ⌈br / M⌉ + ⌈br / bb⌉ (2 ⌈logM–1(br / M)⌉ -1)
```

Joint Operation

- Several different algorithms to implement joins
  - Nested-loop join
  - Block nested-loop join
  - Indexed nested-loop join
  - Merge-join
  - Hash-join
- Choice based on cost estimate
- Examples use the following information
  - Number of records of student: 5,000 takes: 10,000
  - Number of blocks of student : 100 takes: 400

Nested-Loop Join

- To compute the theta join r ⧖ θs

```
for each tuple tr in r do begin
    for each tuple ts  in s do begin
        test pair (tr,ts) to see if they satisfy the join condition θ
        if they do, add tr•ts to the result.
    end
end
```

- ***r*** is called the **outer relation** and ***s*** the **inner relation** of the join.
- Requires no indices and can be used with any kind of join condition.
- Expensive since it examines every pair of tuples in the two relations.
- In the worst case, if there is enough memory only to hold one block of each relation, the estimated cost is
  nr * bs + br block transfers, plus
  nr + br seeks
- If the smaller relation fits entirely in memory, use that as the inner relation.
  - Reduces cost to br + bs block transfers and 2 seeks
- Assuming worst case memory availability cost estimate is
  - with student as outer relation:
    5000 * 400 + 100 = 2,000,100 block transfers,
    5000 + 100 = 5100 seeks
  - with takes as the outer relation
    10000 * 100 + 400 = 1,000,400 block transfers and 10,400 seeks
- If smaller relation (student) fits entirely in memory, the cost estimate will be 500 block transfers.
- Block nested-loops algorithm (next slide) is preferable. (edited)

Block Nested-Loop Join

- Variant of nested-loop join in which every block of inner relation is paired with every block of outer relation.

```
for each block Br of r do begin
    for each block Bs of s do begin
        for each tuple tr in Br  do begin
            for each tuple ts in Bs do begin
                Check if (tr,ts) satisfy the join condition
                f they do, add tr • ts to the result.
            end
        end
    end
end
```

- Worst case estimate: br * bs + br block transfers + 2 * br seeks
  - Each block in the inner relation s is read once for each block in the outer relation
- Best case: br + bs block transfers + 2 seeks.
- Improvements to nested loop and block nested loop algorithms:
  - In block nested-loop, use M — 2 disk blocks as blocking unit for outer relations, where M = memory size in blocks; use remaining two blocks to buffer inner relation and output

```
Cost =   ⌈br  / (M-2)⌉ \* bs + br block transfers + 2 ⌈br  / (M-2)⌉ seeks
```

- If equi-join attribute forms a key or inner relation, stop inner loop on first match
  - Scan inner loop forward and backward alternately, to make use of the blocks remaining in buffer (with LRU replacement)
  - Use Index or Inner Relation if available
- Index lookups can replace file scans if
  - join is an equi-join or natural join and
  - an index is available on the inner relation's join attribute
    - Can construct an index just to compute a join.
- For each tuple tr in the outer relation r, use the index to look up tuples in s that satisfy the join condition with tuple tr.
- Worst case: buffer has space for only one page of r, and, for each tuple in r, we perform an index lookup on s.
- Cost of the join: br (tT + tS) + nr * c
  - Where c is the cost of traversing index and fetching all matching s tuples for one tuple or r
  - c can be estimated as cost of a single selection on s using the join condition.
- If indices are available on join attributes of both r and s, use the relation with fewer tuples as the outer relation.

Observation about B+- trees

- Since the inter-node connections are done by pointers, "logically" close blocks need not be "physically" close.
- The non-leaf levels of the B+-tree form a hierarchy of sparse indices.
- The B+-tree contains a relatively small number of levels
  - Level below root has at least 2* ⌈n/2⌉ values
  - Next level has at least 2* ⌈n/2⌉ * ⌈n/2⌉ values
  - .. etc.
  - If there are K search-key values in the file, the tree height is no more than ⌈ log⌈n/2⌉(K)⌉ thus searches can be conducted efficiently.
  - Insertions and deletions to the main file can be handled efficiently, as the index can be restructured in logarithmic time (as we shall see).

Hashing

Static Hashing

- A **bucket*** is a unit of storage containing one or more records (a bucket is typically a disk block).
- In a **hash file organization** we obtain the bucket of a record directly from its search-key value using a **hash function**.
- Hash function h is a function from the set of all search-key values K to the set of all bucket addresses B.
- Hash function is used to locate records for access, insertion as well as deletion.
- Records with different search-key values may be mapped to the same bucket; thus entire bucket has to be searched sequentially to locate a record.

Hash file organization of *instructor* file, using *dept_name* as key

- There are 10 buckets,
- The binary representation of the ith character is assumed to be the integer i.
- The hash function returns the sum of the binary representations of the characters modulo 10

E.g. h(Music) = 1 h(History) = 2
h(Physics) = 3 h(Elec. Eng.) = 3

bucket 0

bucket 1

15151	Mozart	Music	40000

bucket 2

32343	El Said	History	80000
58583	Califieri	History	60000

bucket 3

22222	Einstein	Physics	95000
33456	Gold	Physics	87000
98345	Kim	Elec. Eng.	80000

bucket 4

12121	Wu	Finance	90000
76543	Singh	Finance	80000

bucket 5

76766	Crick	Biology	72000

bucket 6

10101	Srinivasan	Comp. Sci.	65000
45565	Katz	Comp. Sci.	75000
83821	Brandt	Comp. Sci.	92000

bucket 7

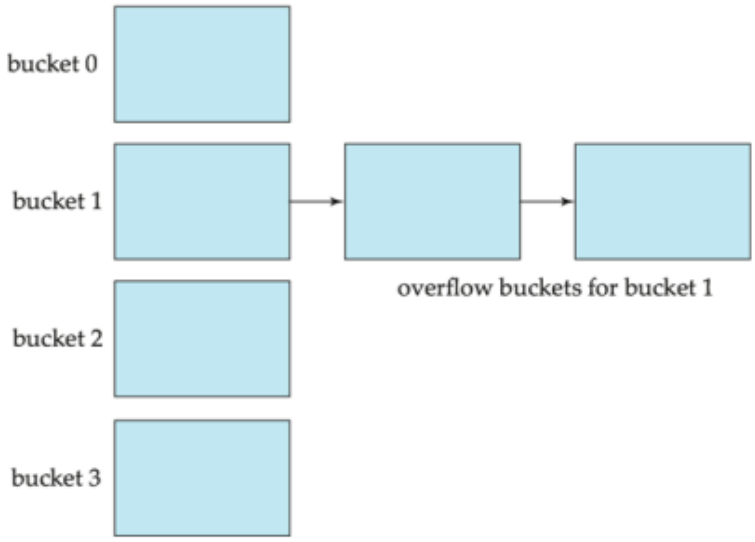
Hash Functions

- Worst hash function maps all search-key values to the same bucket; this makes access time proportional to the number of search-key values in the file.
- An ideal hash function is **uniform**, i.e., each bucket is assigned the same number of search-key values from the set of all possible values.
- Ideal hash function is **random**, so each bucket will have the same number of records assigned to it irrespective of the actual distribution of search-key values in the file.
- Typical hash functions perform computation on the internal binary representation of the search-key.
  - For example, for a string search-key, the binary representations of all the characters in the string could be added and the sum modulo the number of buckets could be returned.

- Best suited for **random access**, bad at **sequential read** (the latter needs ???)

Handling of Overflow Buckets

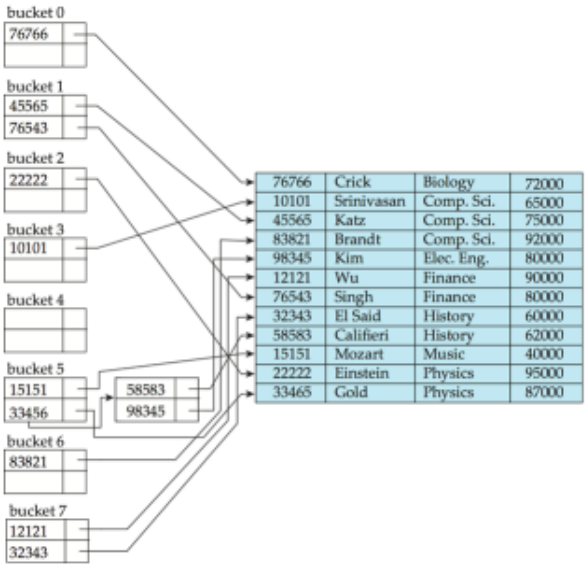
- Bucket overflow can occur because of
  - Insufficient buckets
  - Skew in distribution of records. This can occur due to two reasons:
    - multiple records have same search-key value
    - chosen hash function produces non-uniform distribution of key values
- Although the probability of bucket overflow can be reduced, it cannot be eliminated; it is handled by using **overflow buckets**.
- **Overflow chaining** – the overflow buckets of a given bucket are chained together in a linked list.
- Above scheme is called **closed hashing**.
  - An alternative, called **open hashing**, which does not use overflow buckets, is not suitable for database applications. (edited)



Hash Indices

- Hashing can be used not only for file organization, but also for index-structure creation.
- A **hash index** organizes the search keys, with their associated record pointers, into a hash file structure.
- Strictly speaking, hash indices are always secondary indices
  - if the file itself is organized using hashing, a separate primary hash index on it using the same search-key is unnecessary.
  - However, we use the term hash index to refer to both secondary index structures and hash organized files.

hash index on *instructor*, on *attribute* ID



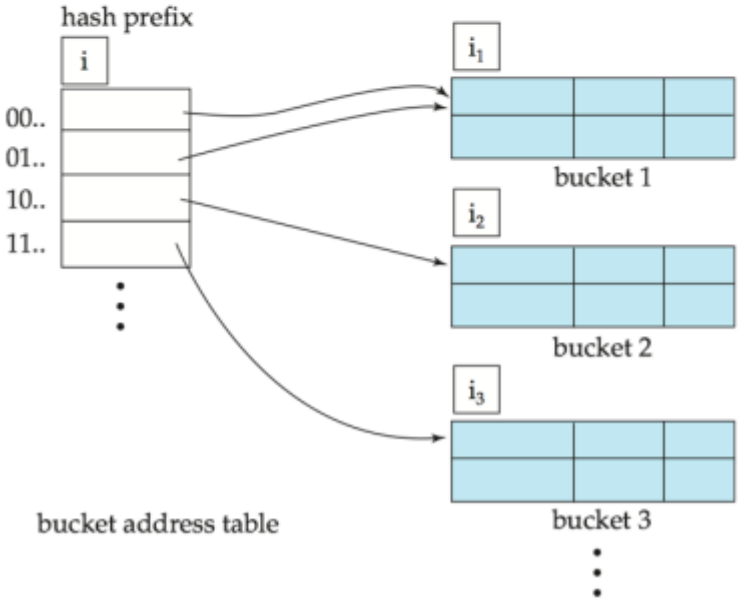
Deficiencies of Static Hashing

- In static hashing, function h maps search-key values to a fixed set of B of bucket addresses. Databases grow or shrink with time.
  - If initial number of buckets is too small, and file grows, performance will degrade due to too much overflows.
  - If space is allocated for anticipated growth, a significant amount of space will be wasted initially (and buckets will be underfull).
  - If database shrinks, again space will be wasted.
- One solution: periodic re-organization of the file with a new hash function
  - Expensive, disrupts normal operations
- Better solution: allow the number of buckets to be modified dynamically.

Dynamic Hashing

- Good for database that grows and shrinks in size
- Allows the hash function to be modified dynamically
- **Extendable hashing** – one form of dynamic hashing
  - Hash function generates values over a large range — typically b-bit integers, with b = 32.
  - At any time use only a prefix of the hash function to index into a table of bucket addresses.
  - Let the length of the prefix be i bits, $0 \le i \le 32$.
    - Bucket address table size = 2i. Initially i = 0
    - Value of i grows and shrinks as the size of the database grows and shrinks.
  - Multiple entries in the bucket address table may point to a bucket (why?)
  - Thus, actual number of buckets is < 2i
    - The number of buckets also changes dynamically due to coalescing and splitting of buckets.

In this structure, i2 = i3 = i, whereas i1 = i – 1 (see next slide for details)



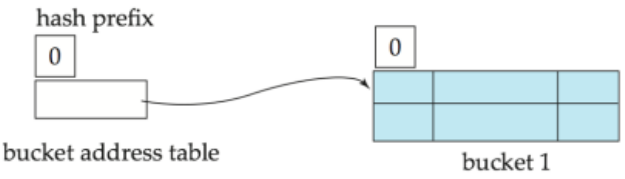
Use of Extendable Hash Structure

- Each bucket j stores a value ij
  - All the entries that point to the same bucket have the same values on the first ij bits.
- To locate the bucket containing search-key Kj:
  - 1. Compute h(Kj) = X
  - 2. Use the first i high order bits of X as a displacement into bucket address table, and follow the pointer to appropriate bucket
- To insert a record with search-key value Kj
  - follow same procedure as look-up and locate the bucket, say j.
  - If there is room in the bucket j insert record in the bucket.
  - Else the bucket must be split and insertion re-attempted (next slide.)
    - Overflow buckets used instead in some cases (will see shortly)

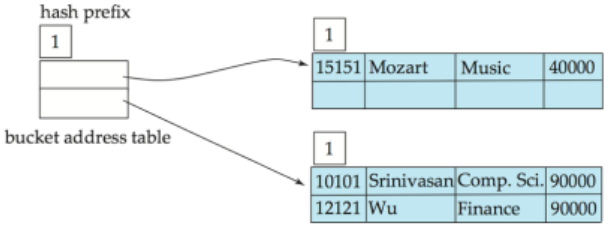
Example:

dept_name	h(*dept_name*)
Biology	0010 1101 1111 1011 0010 1100 0011 0000
Comp. Sci.	1111 0001 0010 0100 1001 0011 0110 1101
Elec. Eng.	0100 0011 1010 1100 1100 0110 1101 1111
Finance	1010 0011 1010 0000 1100 0110 1001 1111
History	1100 0111 1110 1101 1011 1111 0011 1010
Music	0011 0101 1010 0110 1100 1001 1110 1011
Physics	1001 1000 0011 1111 1001 1100 0000 0001

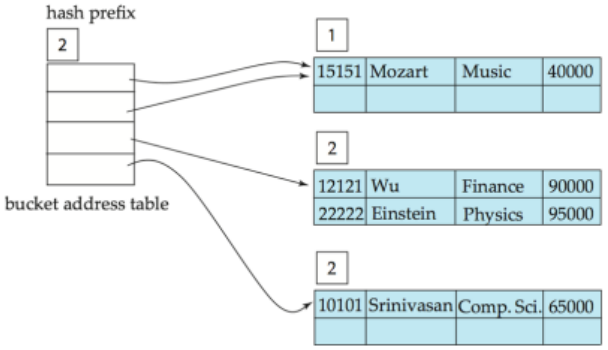
Init Bucket = 2



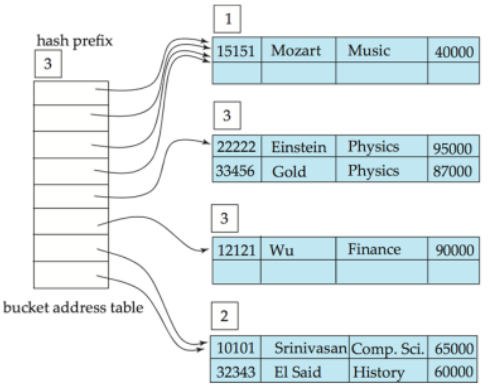
After Insert



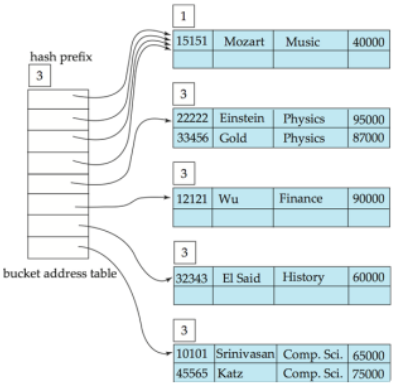
Even more Insert



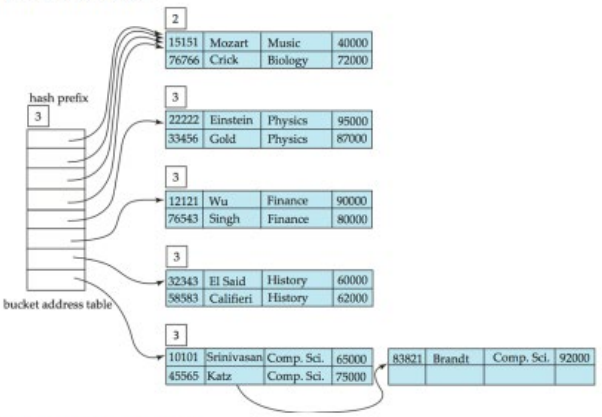
Even, even more insert



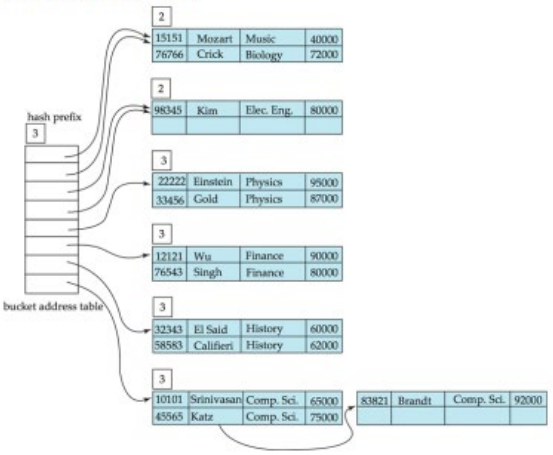
More insert



After 11th Insert



After adding in Kim



Extendable Hashing VS Other Schemes

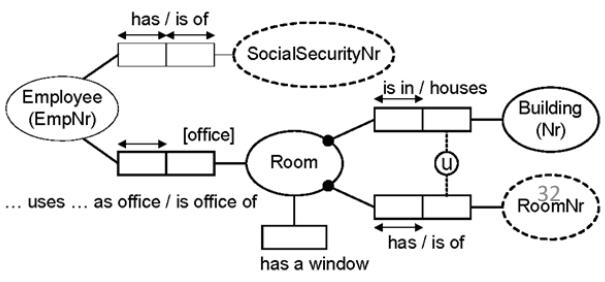
- Benefits of extendable hashing:
  - Hash performance does not degrade with growth of file
  - Minimal space overhead
- Disadvantages of extendable hashing
  - Extra level of indirection to find desired record
  - Bucket address table may itself become very big (larger than memory)
    - Cannot allocate very large contiguous areas on disk either
    - Solution: B+-tree structure to locate desired record in bucket address table
  - Changing size of bucket address table is an expensive operation
- **Linear hashing** is an alternative mechanism
  - Allows incremental growth of its directory (equivalent to bucket address table)
  - At the cost of more bucket overflows

Comparison of Ordered Index and Hashing

- Cost of periodic re-organization
- Relative frequency of insertions and deletions
- Is it desirable to optimize average access time at the expense of worst-case access time?
- Expected type of queries:
  - Hashing is generally better at retrieving records having a specified value of the key.
  - If range queries are common, ordered indices are to be preferred
- In practice:
  - PostgreSQL supports hash indices, but discourages use due to poor performance
  - Oracle supports static hash organization, but not hash indices
  - SQLServer supports only B+-trees
  - **Hashing is usually used to create temporary indice when merging two together**
  - **If not given, assume hash index height as 1. For b+ tree that depends on the formula**
  - **To make Extendable usable for sequential search**, Use Pointer to point to the actual data, rather than directly

Object Role Modelling

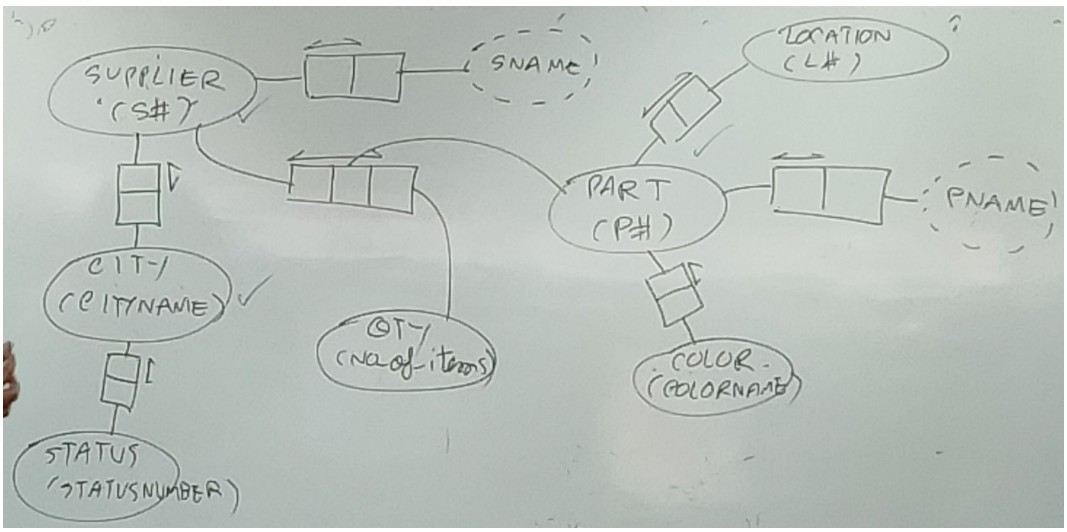
- http://www.orm.net/ ; also see tool **NORMA**
- The **Opposite of ER**
- **Based on 5th Normal Form** (see #isdb)
- Is **a conceptual schema model** invented by Nijssen & Falkenberg (co-author of the Presidential DB textbook in #isdb)
- Design in the Cold War (late '70s) so that the soldiers can easily design their own DB in case the specialists got nuked to hell
  - ER is not easy to point out entity types from an application after all
    - Achieve this by *eliminating attributes*
- Steps:
  - Add in Entity (full line) and Naming/Label (dotted line) type
  - Put "Relevance" relationship type from left->right, and "facts" one from top -> bottom
  - Each Entity type must have a unique identifier -> a selected 1:1 reference type
  - Uniqueness Constraint is put on "Many"(:1 / :Many) side
  - Verb/Role is written inside Uniqueness Constraints
- All of this instantly lands us in **5th Normal Form** (edited)

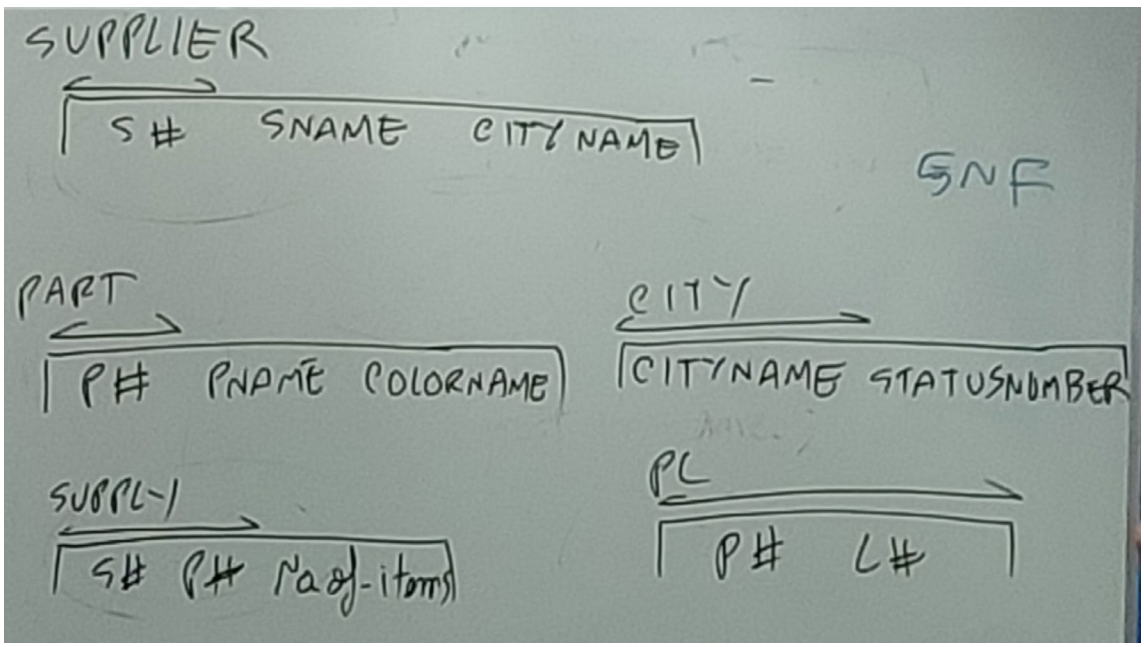


For example, we can turn this ORM schema into 5th Normal Form schema in 3 steps:

```
  - 'Step 1': Mark Binary Relation with the "arrow side" next to it, each one denotes a
"table structure",
        - with the 'primary key' being the 'attributes' inside the bubble the
        - and 'non-key' being 'label on the opposite side'
  - 'Step 2': If Many:Many, split table
  - 'Step 3': If N-nary, split table
```

- All of these can be keyed in Command Line language (Natural Language Interface) as statements ***in English*** without drawing the diagram```
- Each relation is a **deep-structured** (a.k.a. simple sentence; cannot be further broken down/split) **natural language sentences**
    - What we get is technically the 'Optimal 5th Normal Form' (also called 'ONF') -> 5th Normal Form with the Least Table
- Also called a bottoms up approach (go from the smallest ones first that's 5NF, then grouped the common key together) -> the opposite way of how we did ER (edited)





Classic "Religion Example"

CASE STUDY

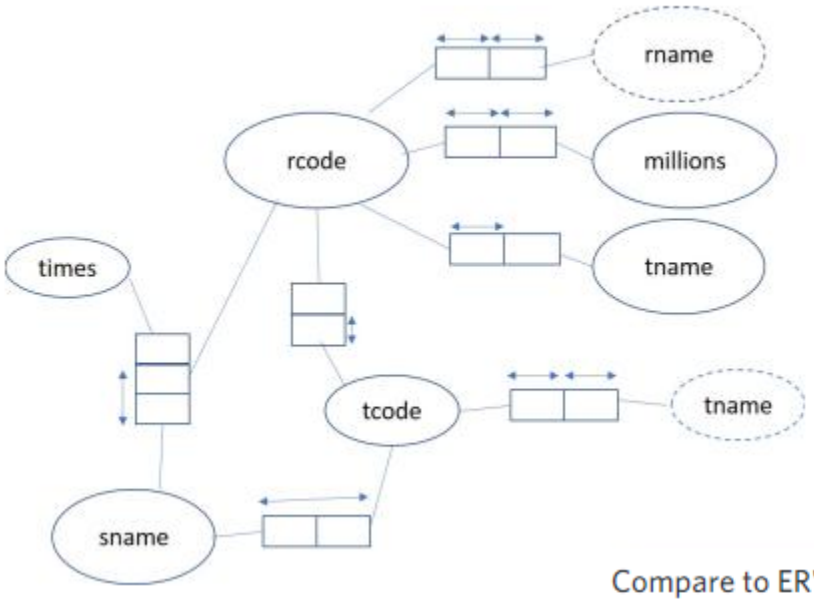
The organizer of an international conference on religions would like to keep information on the conference. Two reports are given. The first report contains information on particular religions. The second one states which speaker speaks on which topics. Furthermore, the organizer would like to use religion codes and text codes instead of names.

You are asked to design 5NF relational database schemata for this conference.

RELIGION_NAME	TEACHER	ADHERENT (MILLIONS)	TEXTS
Christianity	Christ	400	Old Testamant New Testament
Budhism	Budha	300	Sutra
Muslim	Mahamed	400	Koran
Hinduism	Khrisna	350	Upanishad Pakavat kita

SPEAKER	TOPIC	NO. SESSIONS	TEXT READ
hn	Christianinty	2	Pakavat kita
	Hinduism	1	Koran Old Testament
Peter	Budhism	2	Koran
Chandra	Budhism	3	Old Testament
	Christianinty	1	New Testament Sutra

ORM Diagram:



Optimal 5NF from given ORM schema

RELIGION

rcode	rname	tname	millions

TEXT

tcode	tname	rcode

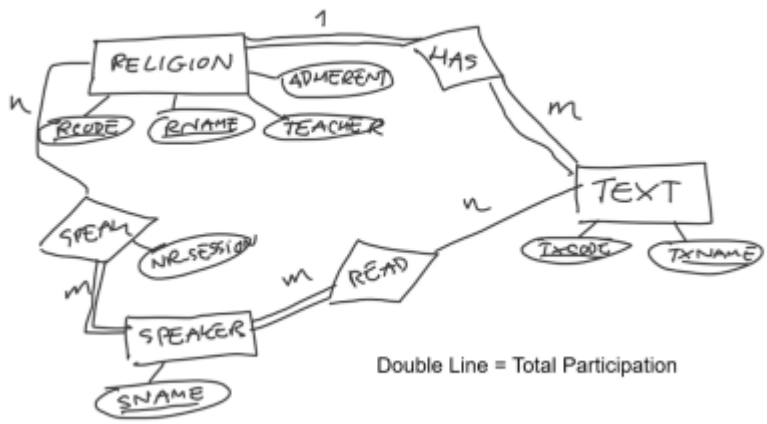
READ

sname	tcode

SPEAK

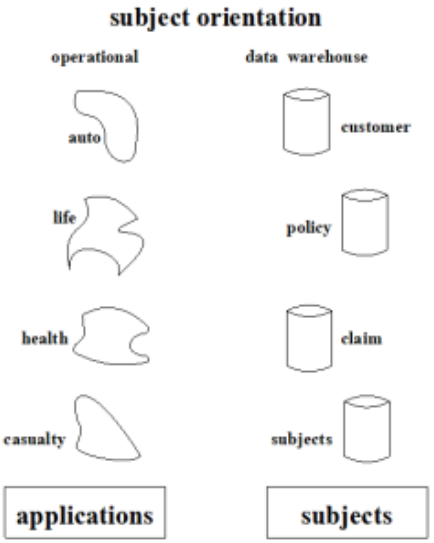
sname	rcode	times

Compare to ER's diagram last semester:

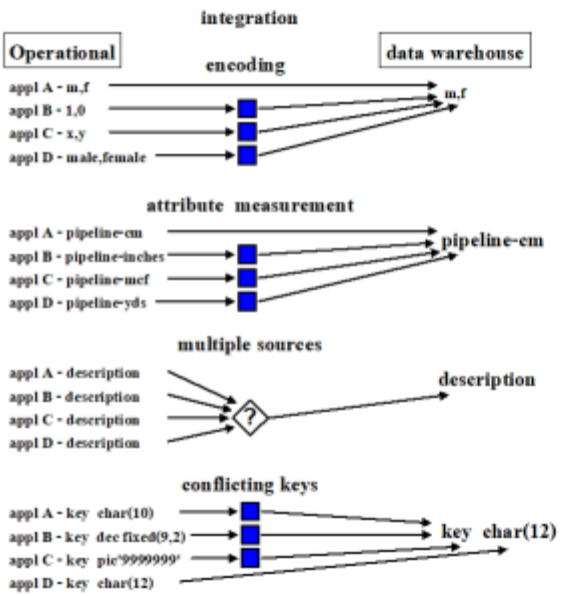


Data Warehouse

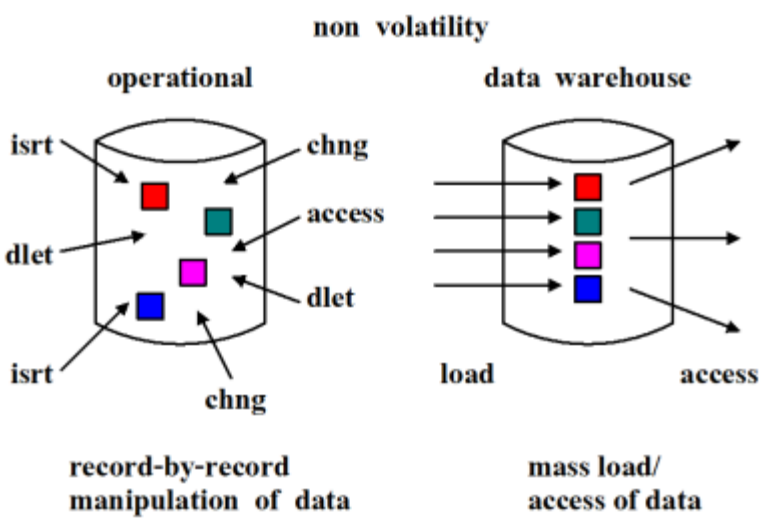
- Data Warehouse is a **Summarized DB for analytics purpose** (only insert/extract/transfer; no update or delete)
- User do operation on "cubes" rather than on SQL directly
- is **Subject Oriented**
- related to **Temporal (Time-related) Database**
- concept developed by Inmon & Kimbal (edited)



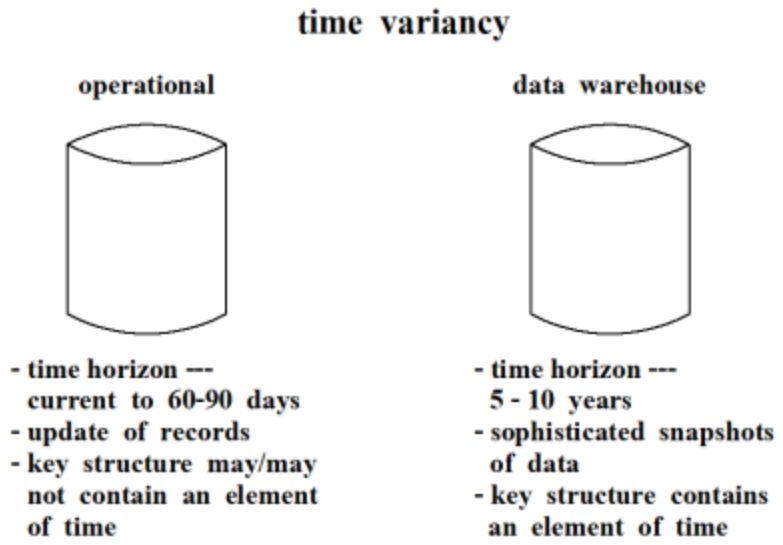
Issue of Integration:



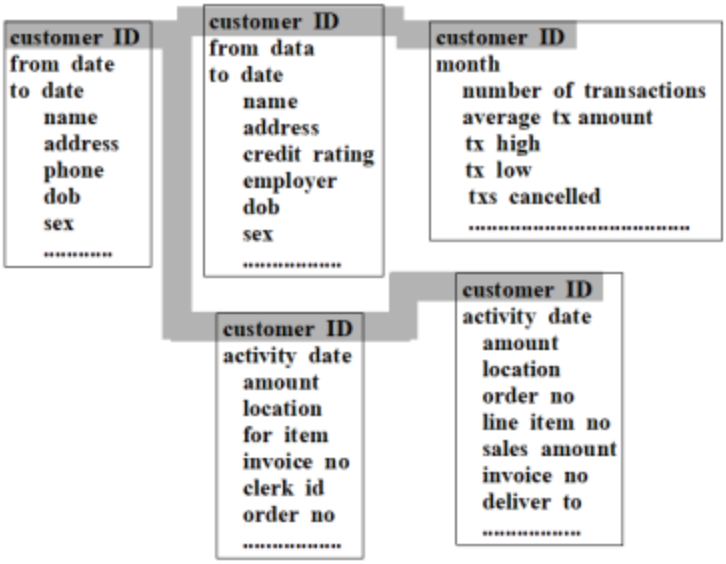
Issue of Non-Volatility:



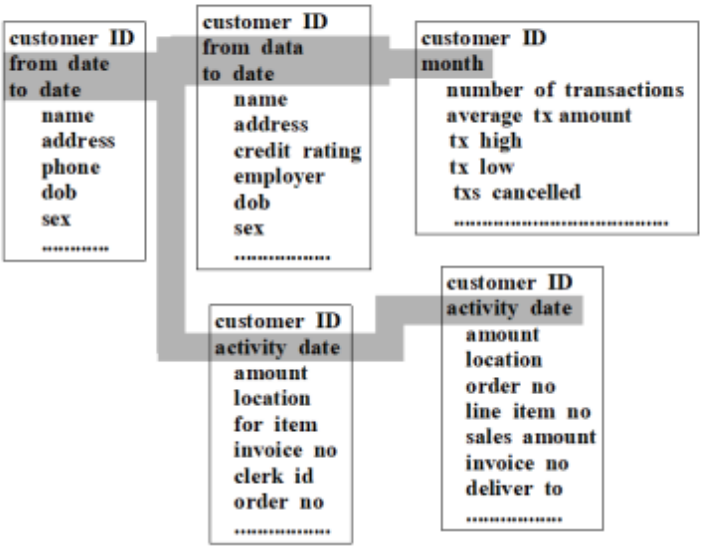
Issue of Time Variances



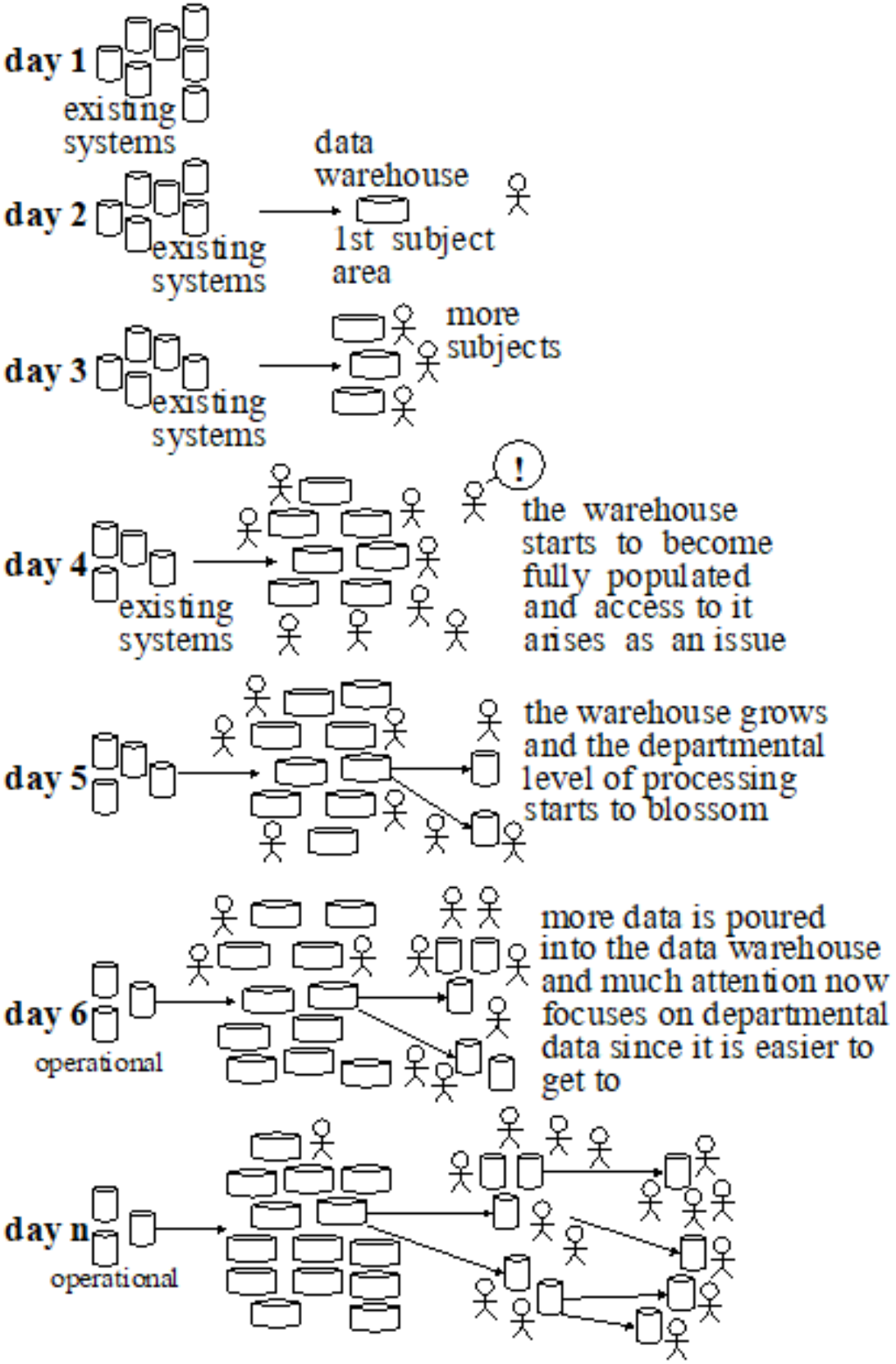
Collection of data in same subject area are collected together using the same common key



Each table in the data warehouse has an element of time as part of key structure, usually the lower part:



Day 1 -> Day N Phenomenon



- Analytics people only concerned about summarized data in the read only mode
- Data Mart is use as subset of Data Warehouse for Department level (edited)

Slowly Changing Dimensions (SCDs)

- Dimension that **stores and manages both current and historical data over time** in a data warehouse.
- is considered and implemented as **one of the most critical ETL tasks** in tracking the history of dimension records.
- 3 Types:
  - Type 1: Overwriting (Default):
    - the new data overwrites the existing data.
    - existing data is lost as it is not stored anywhere else.
    - No additional specifications required
  - Type 2: Create Another Dimension Record
    - retains the full history of values.
    - When the value of a chosen attribute changes, the current record is closed.
      A new record is created with the changed data values and this new record becomes the current record.
    - Each record contains the effective time and expiration time to identify the time period between which the record was active.
  - Type 3: Creating a current value field:
    - stores two versions of values for certain selected level attributes.
    - Each record stores the previous value and the current value of the selected attribute.
      When the value of any of the selected attributes changes, the current value is stored as the old value and the new value becomes the current value.

Data Quality Dimension:

- Some data has a better quality
  - Verified by expert/staff
  - "Out-of-Bound/Uncertain/Verified/..."

WH In Summary

- Practical Solution to Data Integration & Management Problem
- Gives summary, but must ETL (see previous lecture)
- Changing dimension that we should be aware of
- Trying to create a table with 3 dimensions -> "Cube" representation (can have up to 15 or more, but most can only handles 10)
- More than 3 Dimensions possible, but must be SQL (edited)

Temporal Database

- Is a database specialized about time
  - records time-varying information (i.e. Stock Prices)
  - Sometimes does not need to be time-warying
  - Official Definition: **a database which supports some aspect of time , not counting user-defined time**
- Time should be treated as a separate dimension
- Should time be an entity type? (research question)
- Time should be handled by DBMS

Why Can't Object DB replace relational DB?

- Object DB:
  - Don't see table
  - See classes
  - Problem: If we use classes in object orientation, the only way to communicate with classes is though methods
    -> we can't see attributes
  - Cannot do Ad-hoc queries
  - Must create method for each application request -> sucks

Love

```
_____________________________________________
| Person 1 | Person 2 | From Date |  To Date  |
    JOHN       MARY     2020-Nov-1  9999-Dec-3   -> May not be in love forever
                                       NULL     -> Now, the moving row
```

Non-sequenced duplicates: **duplicates in the SQL-sense** where a row exactly matches, column by column, of another now

- Good for querry
- Fck up for Temporal Update/Delete (edited)

6NF (Sixth Normal Form):
- A **Temporal database table** which cannot be splitted (one row, one fact) i.e. ***have only one temporal attribute per table*** (salary, price, amount of item sold, ...)

Exam: Design a temporal database (method: **Do an ORM or ER, then make it 6NF by split of the temporal attributes**)

Multi-Level Secured Database (MLS) -> Different user sees different facts based on the same one

MLS + Temporal Database -> i.e. "Oracle Label Security"

Distributed Database ###```
- **Parallel Database** is now more or less represented in this (since it's the same topic) but are **simply kept close together** in the same room rather being geographically distant
- Usage: Banks, International Companies, ...
- Level of redundancy and what should be done
  -> If there's so much redundancy i.e. 70%, it should be a Centralized DB instead
  -> 1% redundancy works well for distributed DB
- **Definition**:
1) Each Site must be an autonomous system -> computers, DBMS, DB, applications
2) The sites must be interconnected via communication system i.e. Internet -> in the old days this was an issue, and thus mentioned explicitly
3) There are local applications -> use local site's data and resources only
4) There are global applications -> use other site's data and resources

The first 3 alone are called **Distributed Data Processing** (separate from Distributed Database since that needs global application), since each still may not have enough computing power to do certain tasks

Heterogenous vs Homogenous
- **Homogenous Distributed DB**:
  - All sites have identical software
  - Are aware of each other and agree to cooperate in processing user requests.
  - Each site surrenders part of its autonomy in terms of right to change schemas or software
  - Appears to user as a single system
  -> Could also mean they employ the same data model
- **Heterogenous Distributed DB**:
  - Different sites may use different schemas and software
    - Difference in schema is a major problem for query processing
    - Difference in software is a major problem for transaction processing
  - Sites may not be aware of each other and may provide only limited facilities for cooperation in transaction processing
  -> Each site may employ different data model (edited)

Distributed Data Storage
- Assume relational data model
- **Replication**
  - System maintains multiple copies of data, stored in different sites, for faster retrieval and fault tolerance.
- **Fragmentation**
  - Relation is partitioned into several fragments stored in distinct sites
- **Replication and fragmentation can be combined**
  - Relation is partitioned into several fragments: system maintains several identical replicas of each such fragment.

Distributed Database Architecture ## -> THIS WILL BE IN THE EXAM

```
            ['Global User/Application'] -> Have "Fragmentation Transparency" (all it can
see is a Centralized DB), this is the ideal-case
                        ^
                       /|\
                        |
                 Global Database
                        ^
                       /|\
                        |
                    Fragments     ---> ['Fragmentation-level User/Application'] -> Have
"Replication/Location Transparency"
                        ^
                       /|\
                        |
                Replicates/Copies  ---> ['Replicate-level User/Application'] -> Have
"Local Mapping Transparency"
                        ^
                       /|\
                        |
                 Local Mapping    ---> ['Local User/Application']
                        ^
                       /|\
                        |
                 Local Databases
```

At Fragmentation Level:

```
  - Global Database can be fragment into several parts;
      - "Vertical Fragmentation" (by row),
            - Seperate into smaller table for different applications i.e. regional
      - "Horizontal Fragmentation" (by column)
            - Same Application, requires different attribute/part of data i.e. enforced
privacy
      - and "Mixed Fragmentation" (both)
  - Have Replication/Location Transparency
```

At Replicates/Copies Level:

```
  - Replications are used for faster retrieval and fault tolerance
        - Software/Application at this level need to know which copies they are working
with
  - Have "Local Mapping Transparency" (Dont care if the local is graph/object/noSQL or
relational)
```


Fragmentation is hard to implement, and that's why **the current trends is actually moving towards Centralised DB** (edited)

Transaction:

- The common use one is **Two-Phase Commit** (2PC) to ensure **Atomicity** of the Transaction (This also leads to **Consistency**)
  - 3PC can also be use, but is expensive and thus rarely use in practice
  - All sites must be up together for the commit, if one is down then the transaction would fail at the beginning
- The alternative solution is the **Asynchronous Update**
  - All sites don't need to commit altogether, some sites may be down
  - This is also called **Eventual Consistency** (Since eventually, all sites will commit)
    - **The organization which adopts Distributed Database can accept Eventual Consistency anyway**

No organization come up with Distributed Database by design, since it's a huge hassle i.e. a company takeover/merge (edited)

Space intentionally left for extra notes I forgot:

Relevant notes from:

Welcome to #isdb!

Why do we separate database into several tables and join them back when we make queries?
-> No need for useless ***REDUNCANCY***

Redundant (Statement) -> the appearance of the **same fact** (shit that could've been made as its own table) twice or more, Redundant identifier doesn't counts

Fact -> a ground **atomic** predicate which has the truth value "true"

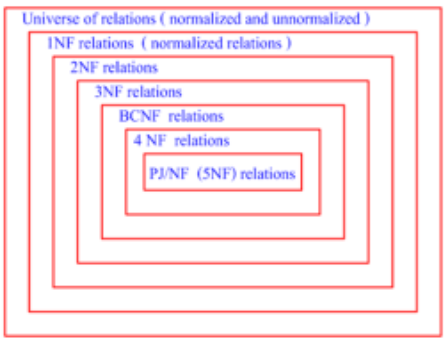
Issue of redundancies in DB:
- **Insertion Problems**
----- **Conflict facts can be inserted** (by human error)
----- **Certain facts** cannot be inserted because they are **against the primary rule** (primary key value cannot be `NULL`)
i.e. Suppliers who didn't supply any parts to the market yet or parts that nobody supplies
- **Deletion Problems**
----- **Loss of fact(s)** that was bundled with another fact (i.e. deletion of `S1 P2` row accidentally remove the fact that `P2` exists)```

Non-Key: Attributes which are not (candidate) keys
Foreign Key (FK): Non-key attributes in a table which is the Primary Key of other tables (edited)

Normalization Process (Analysis, Design?)
To ensure that database tables are good, ready to be used, based on a standard measure (normal form); don't split tables if they are already good (see NF level)

Given: All attributes and relationship between attributes
Output: Relational database schemas (table structure), which guarantees minimum or no redundancy (no insert/delete/update anomalies)

1) Collect all related attributes together to form 1st draft table (1NF)
--- related attributes known from conceptual modelling i.e. entity relationship model
2) Check each table against a higher normal form definition. If passes, do nothing. Otherwise split the table until it conforms the definition.
3) Repeat step 2) until all table agrees with the highest normal form that you understand (**5NF** for this ISDB class, but 6NF exists) (edited)



Normal forms.

0NF/NFNF (Zero Normal Form, Non-First Normal Form):
- still has repeating groups/collection

1NF (First Normal Form):
- if and only if all underlying group contains atomic domain only ('relation' definition); **a proper relational database table**
----- an attributes belong to only 1 domain
----- column values must be atomic
----- no 2 rows are the same
----- rows have no order
- Good for query
- Still bad for insert/delete/update

2NF (Second Normal Form):
- if and only if is **1NF** and **every non-key attributes are fully functionally dependent (FD) on the primary key**

*Non-Key (normalisation only): An attribute which is not part of a candidate key

*FD:
--- i.e. SNAME depends on S# as S# determines SNAME (S#->SNAME)
--- can be both 1:1 or M:1 relation (different S# but same SNAME)
--- Are always given, unless you're data mining

*Full FD: X and Y may be composites as long as Y is fully depends on the whole X and not any of its proper subset i.e. (S#, P#) -> QTY (edited)

3NF (Third Normal Form):
- if and only if is **2NF** and **no FD between non-key attributes** (making each data which depends on other column into its own table)
- i.e. separating

```
| S# | SNAME | CITY | STATUS |
```

into:

```
| S# | SNAME | CITY |
```

and

```
| CITY | STATUS |
```

- Bugs of 3NF:
--- Still has multiple candidate keys
--- Each Candidate Key are Composite Keys
--- Candidate keys are overlapping which each other

BCNF (Boyce-Codd Normal Form; also called 3.5NF):
- If and only if **every determinant is a candidate key** (does no relies on previous steps, works as shortcut)
- All redundancy based on functional dependency has been removed
- divided

```
| S# | SNAME | P# | QTY |
```

into

```
| S# | SNAME |
```

and

```
| S# | P# | QTY |
```

(edited)

4NF (Fourth Normal Form)
- If and only if is **BCNF** and **all MVDS are FDs** (one on the left, one on the right)
- i.e. split the table according to the MVDs from

```
| Course | Teacher | Text |
```

into

```
| Course | Teacher |
```

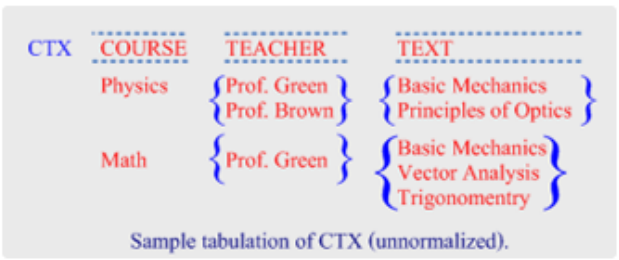
and

```
| Course | Text |
```

Here for example, should one of the professors stop teaching Maths, you wouldn't lose the data on texts for the course
- solves many-to-many (M:M) problems

*MVD (Multi-value dependence):
- One value on the left give set of values on the right, and 2 dependent sets must be independent
- Exists only when a relation has at least 3 attributes

*In case CTX works as primary key for other columns (i.e. CTX+1 random column), don't split them (edited)



Sample tabulation of CTX (unnormalized).

NOTICE: DO NOT SPLIT TABLES IF THEY ARE ALREADY GOOD

Splitability Property of the Table/Lossless Join Property: Shows that R can be split, but may doesn't need to
1. Split R into smaller tables (projections)
2. Join projections back using the natural join
3. If the result of the 2nd step (join) is the original R, then R can be split that way

When should we split table that is known to be splitable? -> When the table is not in the NF we wanted it to be in (there's no insert/delete problem) (edited)

Redundancy -> a fact stored more than once
Fact -> a ground atomic predicate which is true
Predicate -> a function which only returns truth value (true/false; same as boolean)

Fact (here) -> is row that cannot be split

Thus, **5NF** can be rephrase as:
- A table which cannot be split, has only one fact for each row
- A table which can be split, has several facts on each row
- No redundancy issues (Insert/Delete/Update problem)

5NF (Fifth Normal Form)'s working definition
- If R cannot be split -> R is in 5NF
- If R can be split, and the small table after the split (projections) have at least one candidate key of R -> R is in 5NF

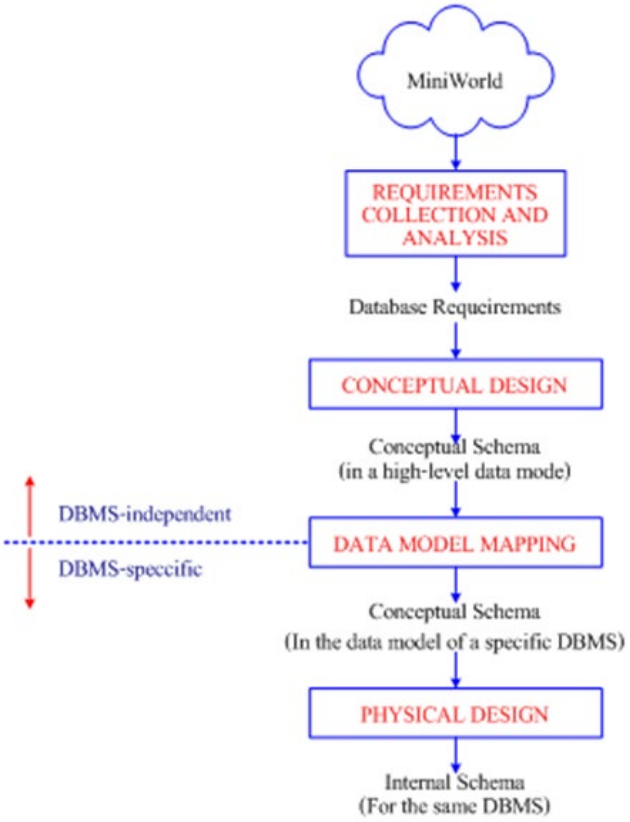
5NF (Fifth Normal Form)
- If and only if **R is equal to join of projections** (does not relies on previous steps at all)
- Is actually the easiest one lol 
- It's very rare that 4NF doesn't conforms to 5NF (edited)

Join Dependency (JD)
- The splitability property of a function (so it's not that rare)
- Can be split if having JDs, cannot if didn't have JD
- If and only if **R is equal to the join of its projections** on X, Y, ..., Z where X, Y, ..., Z are subsets of the set of attribute of R; **a consequence of a candidate key of R**
- Generalization scale (more -> less): **JD** >> **MVD** >> **FD**
- **JD**s are the most general form of dependency possible

Cyclic dependency are rare forms of JD (see SPJ) (edited)

SPJ is the join of all three of its (binary) projections but not of any two.

DB design steps (simplified ver.) -> based on usage (edited)

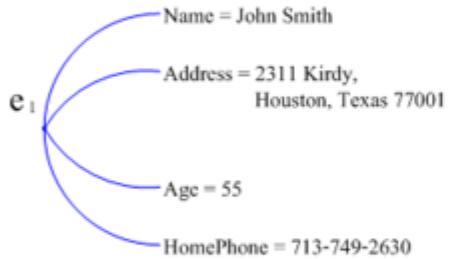


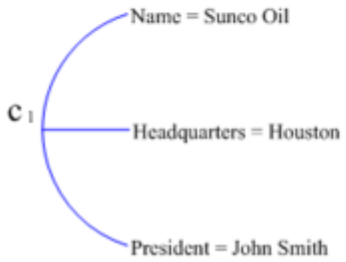
Both ER and Relational DB can be called conceptual schema, but at different level (RDMBS shit should be called "Logical Schema/Logical level" to avoid confusion with ER) (edited)

Entity-Relationship Model (ER Model): Peter P. Chen/Pin Shan Chen (1974)

Entity: an object of interest (within an application model)
Attribute: are properties of entities
Relationship (in ER model; The one in Relational DB is Relation): association between entities

See Example below (works with both physical and logical stuff as shown in picture): (edited)

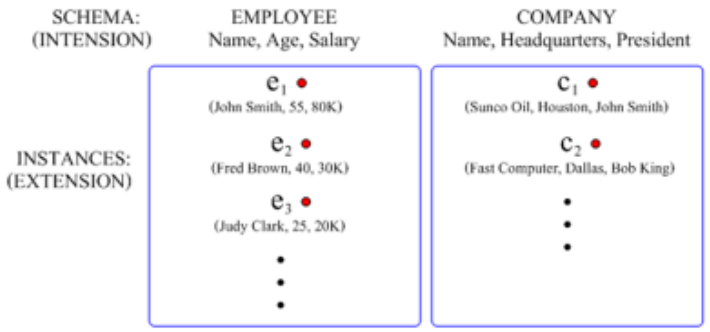


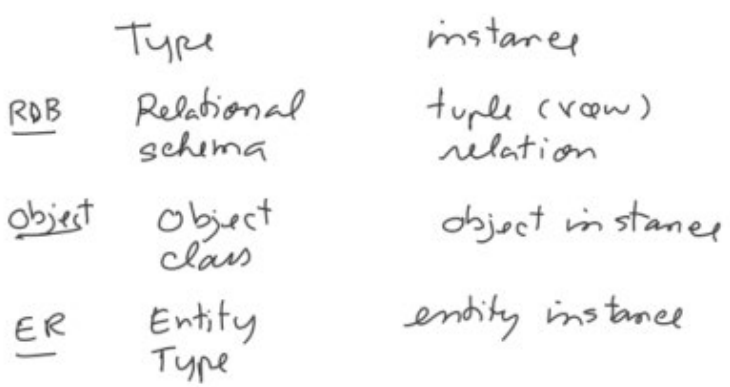


Hierarchy of composite attributes

Two or more entities can be group into entity types (in which we use to design DB); **Don't mix Entity Types and Entities together**

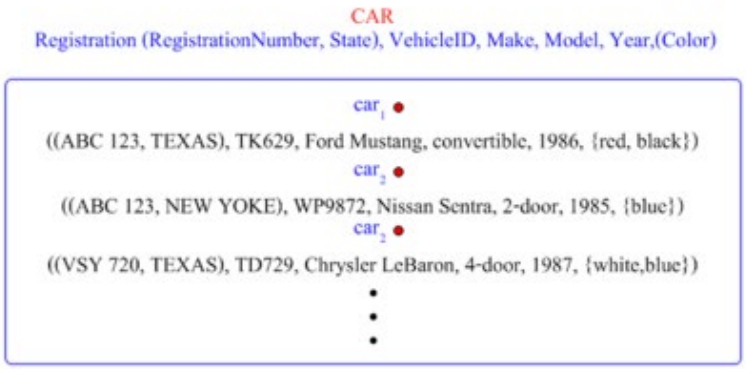
Entity Types -> Table name
Instances -> Tuples/row relation in Relational DB (edited)



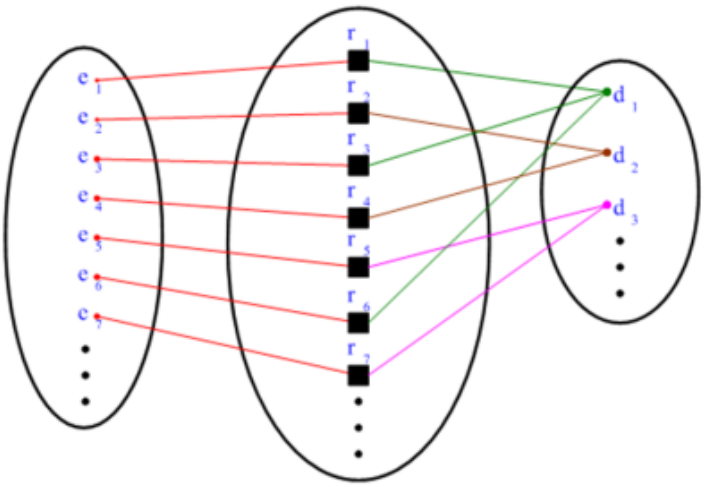



```
Multivalued attributes are shown between set braces { }.
Components of a composite attribute are shown between parentheses().
```

Note that these are not allowed in Relational DB model, but allowed in ER model (edited)



Relationships between employees and companies (r1, r2, r3, ...) are called **relationship instances**, EMPLOYEE on the left and COMPANY on the right
Note that **ENTITY TYPES** use capital letters, while **entities** use lower case ones (edited)

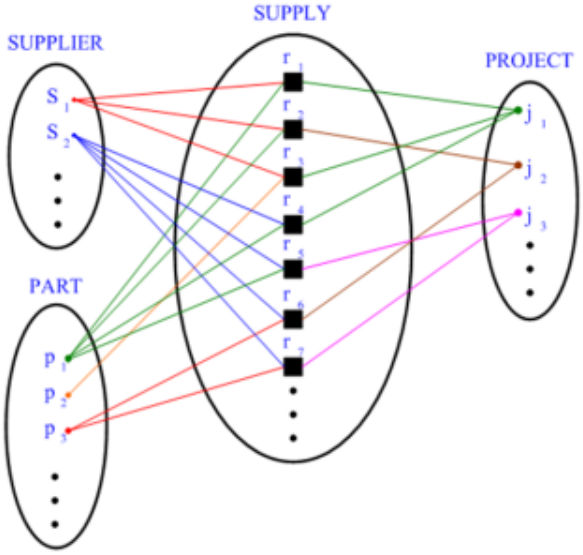


Clarification
- **Many-to-One**: Many people works for one department
- **One-to-One**: Each person only works for one department
- **One-to-Many**: Each person may work for many departments
- **Many-to-Many**: Many people works for many departments, each person can work in >1 departments

Thus the example above is Many-to-One participation (edited)

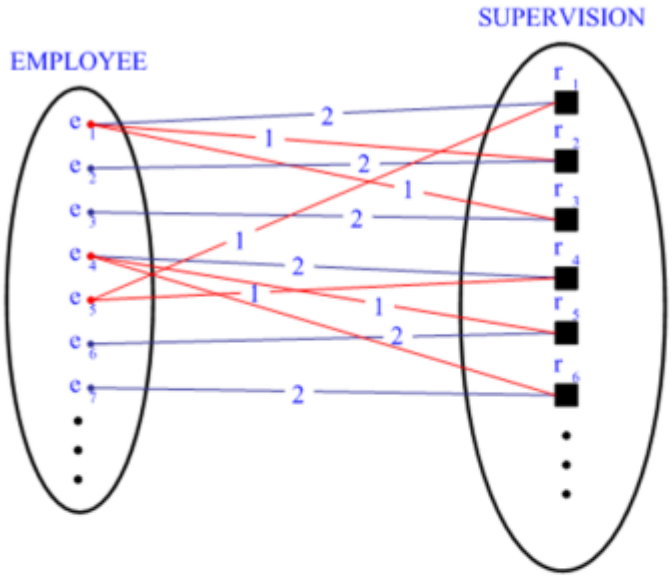
ER model lacks relationship between attributes, nor does UML diagram - > Thus we cannot claim 3rd NF straightaway from just ER model yet (edited)

Many-to-One example for ER:
As per shown, ER supports ternary relationship type (already an N-nary relationship type) (edited)

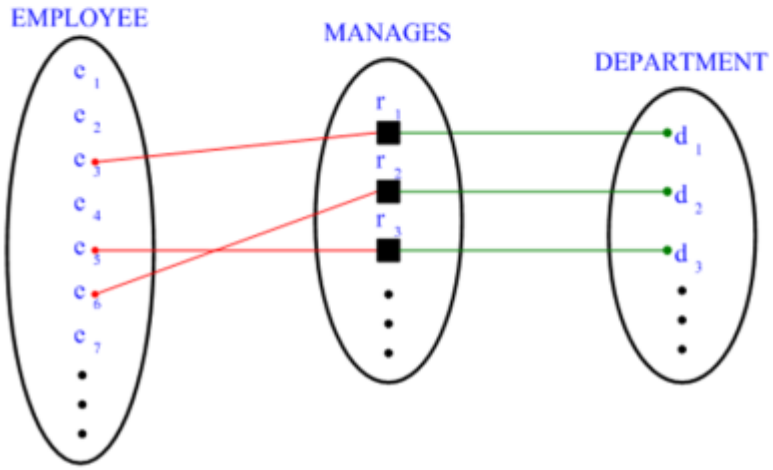


Recursive Relationship example (i.e. one employee is a supervisor of other employees; many supervisors can be under the higher tier supervisees) -> in fact a Many-to-One relationship

According to the diagram 1-> supervisor, 2-> higher supervisor (edited)



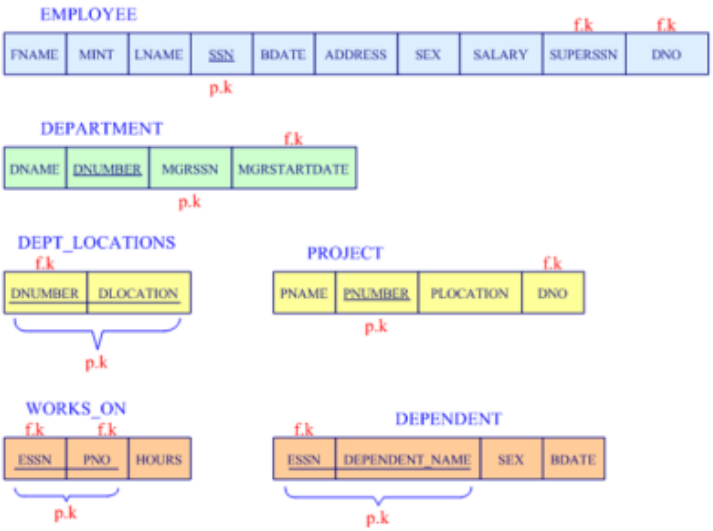
One-to-One (1:1) relationship example:



ER schema diagram (also called Peter Chen model):

- Diamond -> Relationship types
- Rectangle -> Entity types
- Oval -> Attributes
- M, N, 1 -> Cardinalities, M and N are both many and work exactly the same (edited)

Relational DB schema corresponding to ER model above:

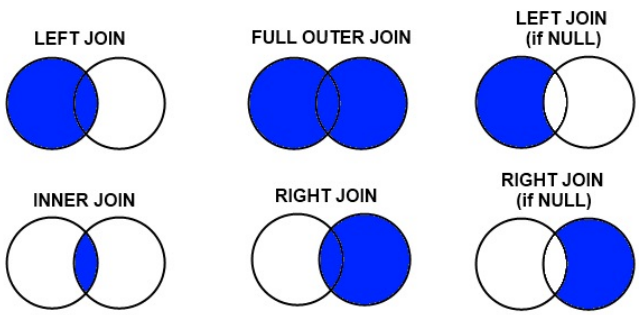


Nowadays, it's widely accepted that you need to work on ER model first for conceptual modelling **before** working on the table themselves (edited)

Entity Types: -> a collection of object of interest, shit people care i.e. customer @convenient stores may not be entity types, but members are
1. Noun with Identifiers
Verbs which are use as nouns, with Identifiers

2. There should be some attributes or the object is related to >1 other Entity Types

3. Do not confuse **Entity Types** with **Reports**
Example 1 Transcript is a Report because it can be derived from Entities, and thus not an Entity Types - however, it's possible to treat the transcript as an Entity Type if they want to track the document itself (who sign the transcript, cost, requester, issue date, ...) (edited)



What's a ***relation*** in database? (possible midterm exam) -> ***rows of a table (without the headings)***
Also: **subset of the Cartesian product of domains**

Back in highschool: **a set of n-tuples** (pretty much a row in table for database) **of a Cartesian product** (a set of all relations/ordered pairs) (edited)

Database Models
--- **Represents Data and Relationships**
--- **Provides Database Language(s)** to work on the database relationship

Relational DB Model -> **Relation** to represent data and relationship (the DB Model)
--- In Database, we only keep row/tuple that is `True`; or else the query would be too complex

Relation (again) -> **subset of the Cartesian product of domains**
------ **Cartesian Product** -> **Concatenation/Combination of all possible domain values**
------ **Domain** -> **a set of permuted values**

A **Table** is the most popular representation of a relation for software development
Table which represent relations must have the following properties:
--- 1). **An attribute refers to only one domain** (one column: one topic)
--- 2). **Column values must be atomic** (1 pieces, cannot be further broken down)
--- 3). **No 2 rows are the same** (according to the set theory; having them brings no new info)
--- 4). **Rows have no order**

The order of the values in a tuple does matters
SQL works when the input is a relation and the output is also a relation
Don't try to use SQL to work with crossmap (edited)

Data(base) Model comprised of 3 characteristics:

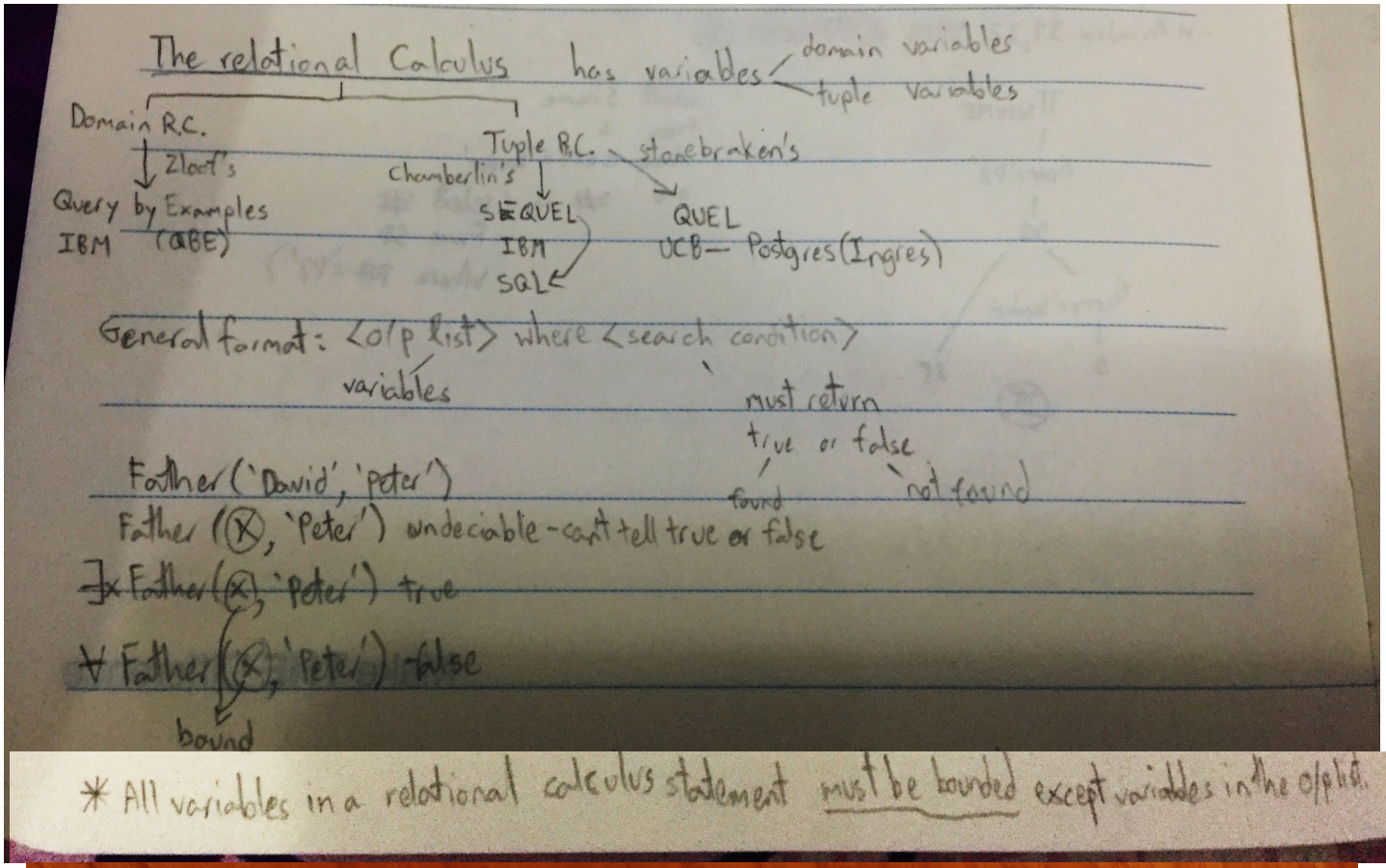
1).**Data Structure** -> logical data structure to **represent objects of interest and relationships among them** (as seen by application developers and users)

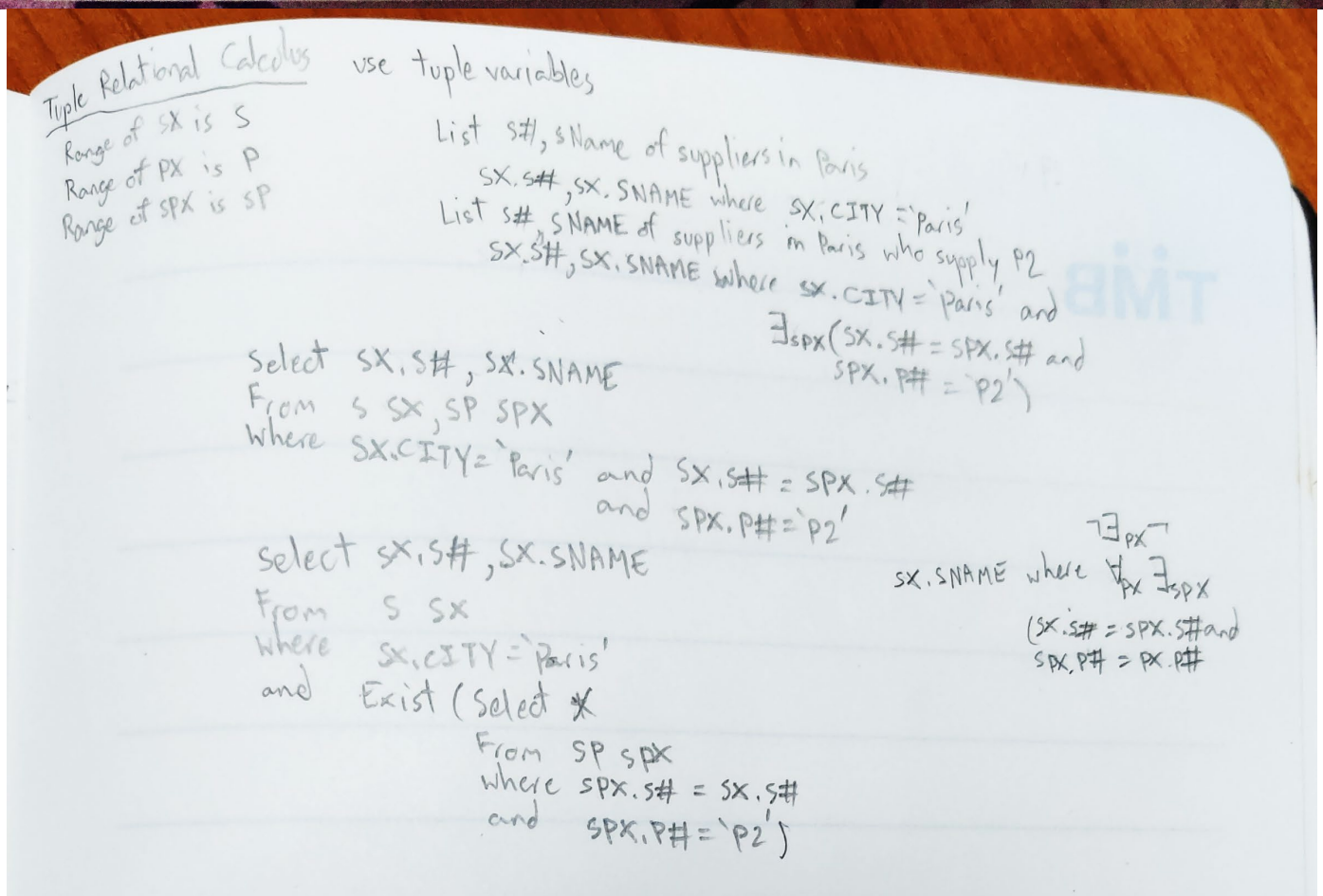
2).**Integrity Constraints/Rules** (also called **Data Integrity**) -> **rules that enforce correctness of the logical data structure**
- **Entity Integrity** -> **Primary Key** must not be `Null`
--- *Key* -> *identifier*
--- *Superkey* -> an *attribute/set of attribute* whose values **uniquely identifies a tuple** (row on table)
--- *Candidate Key* -> **smallest *superkey***; can uniquely identify any database record without referring to any other data; each one is a *candidate* for primary key
--- *Primary Key* -> only one candidate key can be primary key; the remaining CKs are called *Alternate Key*
- **Referential Integrity** -> **Foreign Key** must match primary key values (or be `Null`)

Superkey -> Candidate Key -> Primary Key
Key is logical, *Index* (an access mechanism) is Physical
`Null` is usually 1) *attribute not applicable* or 2) *values unknown*, **but shouldn't be both**
Meaningful digits(keys) -> ensures the *uniqueness of the values*; but isn't needed nowadays and thus should be avoid at all cost; also avoid using those whose can be changed later as key
Course Goal: ***Same Objects Must Use Same Identifiers***

3).**Data Manipulation (Language)** -> **Languages which are used to retrieve and manipulate the logical data**
- **Relational Algebra** (or Relational Calculus equivalents i.e. **SQL**) : use by the query optimizer of the DBMS, not us
-----> *Union, Intersection, Difference, Product, Select, Project, Join, Divide*
- **Relational Complete Language(s)** -> Language that **can fully represents** the Relational Algebra
- **Relational Assignment** (edited)

#database-system! Extras





Tuple Relational Calculus — use tuple variables

Range of SX is S
Range of PX is P
Range of SPX is SP

List S#, SName of suppliers in Paris
SX.S#, SX.SNAME where SX.CITY = 'Paris'

List S# SNAME of suppliers in Paris who supply P2
SX.S#, SX.SNAME where SX.CITY = 'Paris' and
$\exists_{SPX}$(SX.S# = SPX.S# and SPX.P# = 'P2')

```
Select SX.S#, SX.SNAME
From S SX, SP SPX
Where SX.CITY='Paris' and SX.S# = SPX.S#
                      and SPX.P#='P2'
```

```
Select SX.S#, SX.SNAME
From   S SX
where  SX.CITY='Paris'
and    Exist (Select *
              From SP SPX
              where SPX.S# = SX.S#
              and   SPX.P# = 'P2')
```

$\neg\exists_{PX}$ SX.SNAME where $\forall_{PX}\exists_{SPX}$ (SX.S# = SPX.S# and SPX.P# = PX.P#)

Domain Relational Calculus (DRC) employs domain variables → QBE (Query By Examples)

S [S# SNAME CITY STATUS]
P [P# PNAME COLOR]
SP [S# P# QTY]

List SNAME of London suppliers who supply red parts.

SNAMEX (domain variable) where $\exists_{SX}\exists_{PX}$ S(S#: SX, SNAME: SNAMEX) and SP(S#: SX, P#: PX) and P(P#: PX, color: 'Red')

Query By Examples (QBE)

List SNAME of London suppliers who supply red parts.

S [S# SNAME CITY]
_SX P.SNAMEX London

SP [S# P#]
_SX _PX

P [P# COLOR]
_PX Red

examples

$\sigma^{S}_{CITY='London'}$ — selection on non Linear

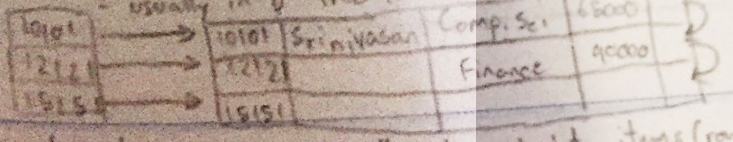
$\sigma^{S}_{S\#='S99'}$ — Selection on key

Index scan - search algorithms that use index

Primary index - The search key of primary index is not necessary primary key
- also called clustering index

Secondary index - non-clustering index

Dense index - Index record appears for every search-key value in the file
- sorted
- usually in B^+ tree format



Make THE Difference

in the same or adjacent database block

Cluster (in physical database): logically adjacent data items (rows, etc.) are stored together physically

multiple clusters? no, but possible

TMB Distributed Databases

by Ceri, Pellagatti, ~1980

Relational Algebra

in SQL, data types must be union compatible

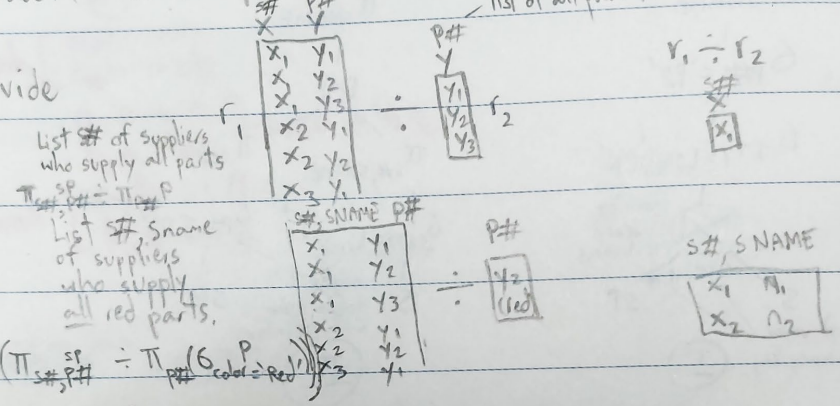
Union $r_1 \cup r_2$

intersection $r_1 \cap r_2$

Difference $r_1 - r_2$

Product (The Cartesian product)

Divide



$\pi_{S\#,SNAME}(S \bowtie (\pi_{S\#,P\#}^{SP} \div \pi_{P\#}(\sigma_{color='Red'}(P))))$

Exist $\exists$ | All $\forall \leftrightarrow \neg\exists\neg$

The relational Calculus has variables — domain variables / tuple variables

Domain R.C. → Zloof's → Query by Examples (QBE) IBM

Tuple R.C. — Chamberlin's → SEQUEL, IBM → SQL

Tuple R.C. — Stonebraker's → QUEL, UCB — Postgres (Ingres)

General format: <o/p list> where <search condition>

variables

must return true or false

Father('David', 'Peter') found

Father(X, 'Peter') undeciable - can't tell true or false not found

$\exists x$ Father(x, 'Peter') true

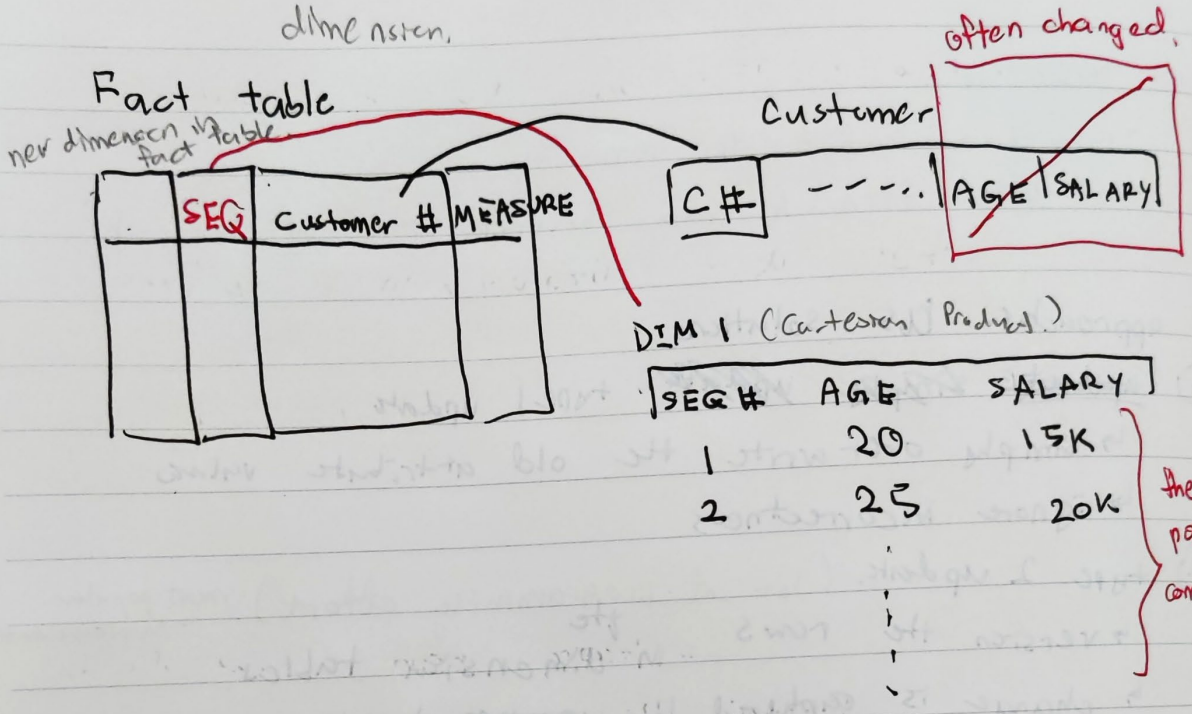
$\forall$ Father(x, 'Peter') false

bound

* All variables in a relational calculus statement must be bounded except variables in the o/p list

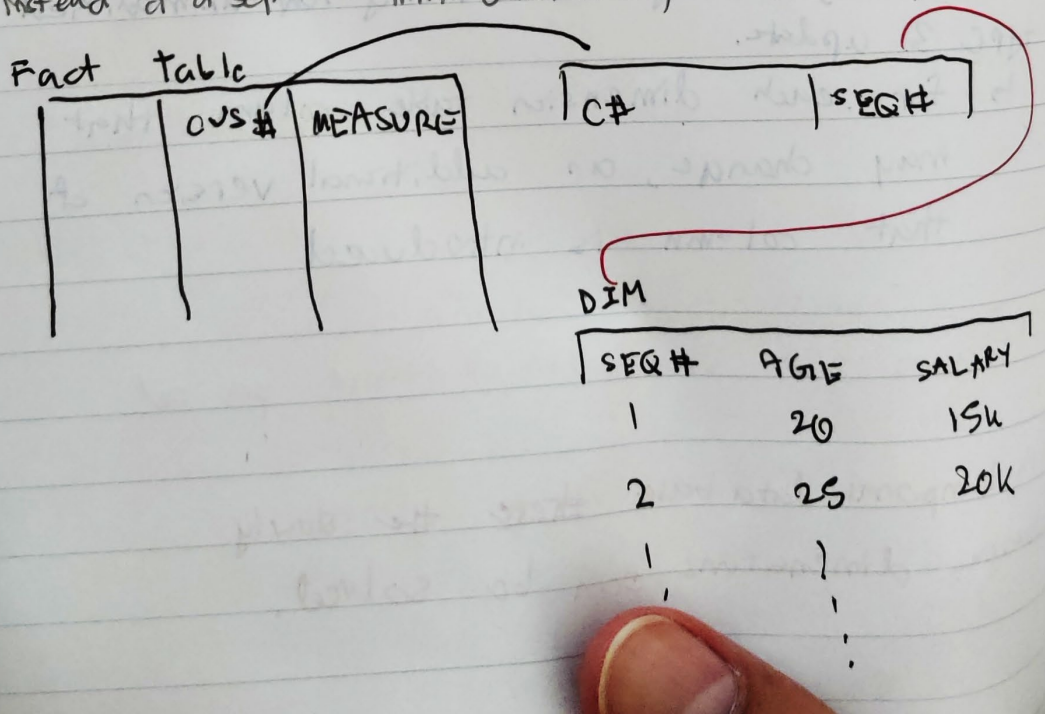
Mini dimensions

↳ seperate changed things to seperate dimension.



Out triggers. (like snow flakes)

↳ Instead of a seperate mini dimension, foreign key on customer



challenge to data warehouse system.

NO.
DATE / /

Slowly Changing Dimensions (SCDs)

have values

what if attribute value changes.

Sale person John
- ↳ move from Team A to Team B.
- ↳ all sales from A team because sales at team B.

3 approaches as solutions:

(1) ~~update simply~~ type 1 update
- ↳ simply overwrite the old attribute values
- ↳ ignore incorrectness

(2) type 2 update. (lot of programming effort) most popular implementation
- ↳ version the rows in dimension tables.
- ↳ change is captured by ~~a new~~ inserting a new row with the updated attribute values, leaving the existing row unmodified

(3) type 3 update.
- ↳ for each dimension table colum that may change, an additional version of that column is introduced
  - ↳ one version per column.
  - ↳ does not show valid time period.

★ With temporal database ~~there~~ the slowly changing dimensions can be solved.

DATE

Degenerate Dimensions.

↳ possible to not have a dimension table for a dimension

↳ in fact table there can be a dimension w/o a dimension table.

— : dimension

refer to tx of operational database.

Book ID	City ID	Day ID	Transaction ID	Sale	Price
			↑ no dimension table.		

Junk Dimension (Miscellaneous Dimension)

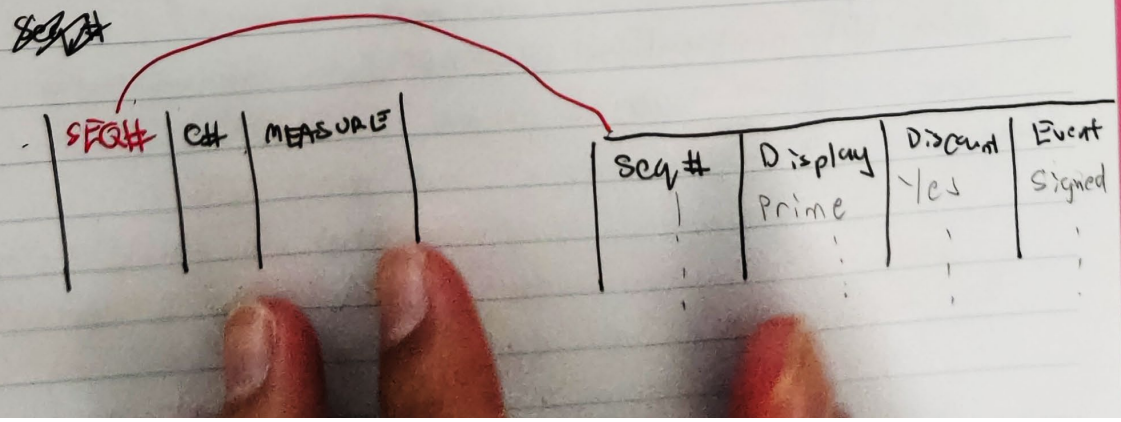
↳ attribute display: in prime location, secondary, not display.

1. discounted, not discounted

↳ not dependent to each other

↳ unrelated attributes

↳ create a dimension for the attributes.



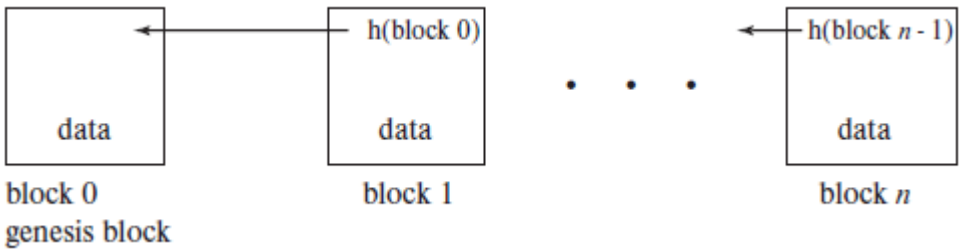
#database-system! Blockchain Moment

> What's Blockchain
> (Textbook Answer: *Database System Concepts 7th Ed.*, Ch.26, Page 1254)

- Blockchain is **a linked list of blocks of data**, in which the pointers in the linked list include **not only** the identifier of the next older block, but also a hash of that older block.

> How does Blockchain works?
> (Textbook Answer: *Database System Concepts 7th Ed.*, Ch.26, Page 1254-55)

- **Genesis Block/Block 0** is created by creator of the chain
- **Each time a block is added** to the chain, it **includes the pair of values** (pointer-to-previous-block, hash-of-previous block).
- Any **change made** to block is easily **detected by comparing a hash of that block to the hash value contained in the next block in the chain**.
  -> to tamper hash, you need to also change the previous and subsequent ones : thanks to the ***Hash-Validated Pointer*** above
- Hash function must have certain **mathematical properties** to make it **easy to detect tampering**
  -> **Nonce** (Number Only Used Once) is added to block so that, when rehashed, meets the difficulty level restrictions.
- Chain itself is **replicated and distributed** among many independent **nodes** so that **no single node or small group of nodes can tamper with the blockchain**.
- A **distributed consensus algorithm is needed** to **maintain agreement** on correct current state of Blockchain among the nodes.
  -> A variation of Algo. exists between **Robustness** (against attacks) and **Performance** (Latency & Throughput) for different usage (edited)



We summarize this discussion by listing a set of properties of blockchains:[2]

- **Decentralization:** In a public blockchain, control of the blockchain is by majority consensus with no central controlling authority. In a permissioned blockchain, the degree of central control is limited, typically only to access authorization and identity management. All other actions happen in a decentralized manner.
- **Tamper Resistance:** Without gaining control over a majority of the blockchain network, it is infeasible to change the contents of blocks.
- **Irrefutability:** Activity by a user on a blockchain is signed cryptographically by the user. These signatures can be validated easily by anyone and thus prove that the user indeed is responsible for the transaction.
- **Anonymity:** Users of a blockchain have user IDs that are not tied directly to any personally identifying information, though anonymity may be compromised indirectly. Permissioned blockchains may offer only limited anonymity or none at all.

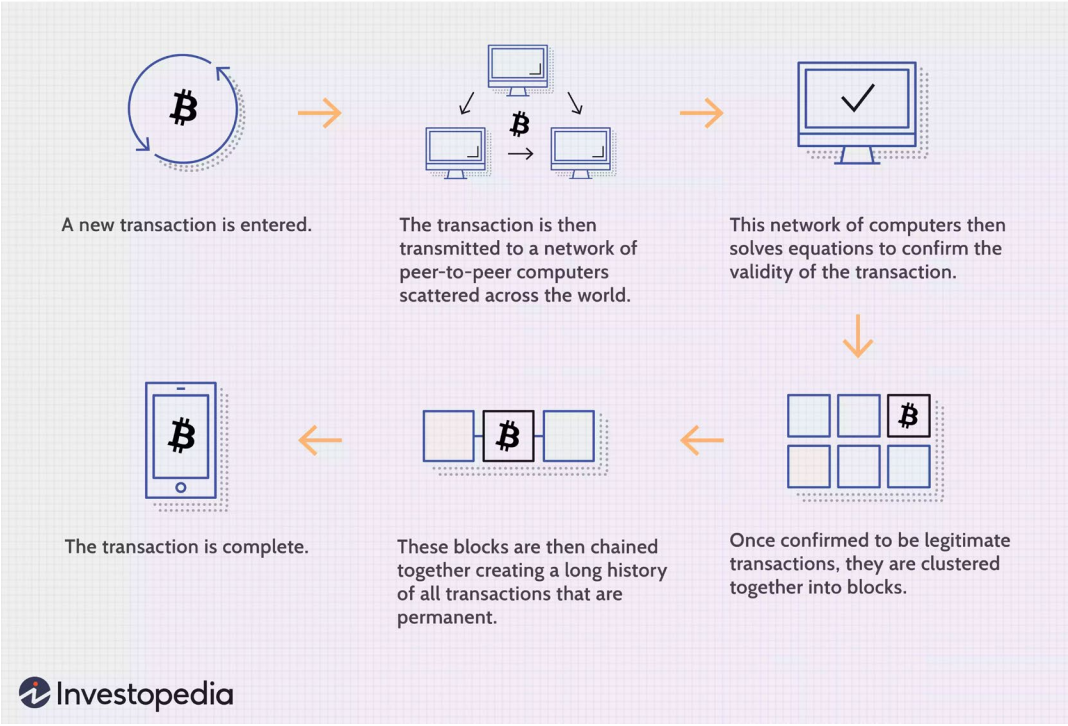
#database-system! Blockchain Moment

> How does Blockchain works? (Investopedia)

- Blockchains **store data in blocks** that are then **chained together**.
- Once a **new block is filled with data**, it is **chained to the previous block. Then**, This new information is compiled into a newly formed block.
- **Each node** on a Blockchain has **access to the whole Blockchain**.
- **Each node has a full record** of the data that has been stored on the blockchain **since its inception**.

*Note: Blockchain is used in decentralized way, but is possible to be centralized.

Transaction Process:



> Blockchain vs Relational Database
> (Textbook Answer: *Database System Concepts 7th Ed.*, Ch.26, Page 1276)

- In traditional (relational) database systems, **Fault-Tolerance** is measured by the performance of the recovery manager (recovery time).
- A blockchain system, in contrast, uses a consensus mechanism and a replication strategy designed for continuous operation during failures and malicious attacks.
- Therefore, besides **measuring throughput and latency**, one must also measure how these **performance statistics change in times of failure or attack**.

- **Scalability** is a performance concern in any distributed system, in which blockchain is one of its member by design.
- Under Byzantine consensus, every transaction needs the agreement of a majority of the non-failed nodes,

-> the **number of nodes** that must agree **not only starts much larger but also grows faster as the network scales.**``` (edited)

#database-system! Blockchain Moment

> Blockchain vs Relational Database
> (Textbook Answer: **Gupta, M.** (2018). *Blockchain for dummies* (IBM Limited ed.).)

While the blockchain contains transaction data, it's **not a replacement for databases**, messaging technology, transaction processing, or business processes. Instead, the blockchain contains verified proof of transactions. However, while blockchain essentially **serves as a database for recording transactions**, its benefits extend far beyond those of a traditional database. Most notably, it **removes the possibility of tampering by a malicious actor** (for example, a database administrator).

It's impossible to compare **Blockchain**, a ***data structure***, which is on *Physical level* with **Relational Database**, a ***Logical Model***, which itself is on the *Logical Level*.

However, a comparison between Traditional DBMS itself and Blockchain is still possible. **DBMSs**, both SQL and noSQL often utilize the **B-tree data structure*** and its variations (B+-tree) to implement indexing which allows for fast and efficient look up. While a **Blockchain**, in essence, is a secure linked list, which is a long-trailing one-dimensional data structure, unlike B-tree. (edited)

> Blockchain vs Relational Database
> (Textbook: **Garcia-Molina, H., Ullman, J. D., & Widom, J.** (2014).
> *Database systems: the complete book.*)

SQL and noSQL DBMSs internal data structure tends to vary depending on the DBMS, but the most important data structure for DBMSs is the B-Tree.[6]

B-tree was invented to store large amounts of data for retrieval. Its main characteristic is its high branching factor with shallow and wide structure, meaning it will typically have a single-digit height even with millions of entries. [7] Which ensures that disk reads are kept as few as possible during data search and retrieval. [8]

A Blockchain, on the other hand, has a structure similar to that of a linked list. Each block is identifiable by its cryptographic hash as a pointer. Blockchain generally has a very long one-dimensional structure as a result. However, all the transaction data are efficiently kept as a summary in a Merkle tree inside the body of a block since all blocks must have the information of other transactions. [9] (edited)

#database-system! Blockchain Moment

> Applications which Blockchain is unsuitable for?

These 3 characteristics are tasks Blockchains are unsuitable for:

1. **Isolation Usage:**
   If the system is used within only one organization without external stakeholders, a **centralized** system is more suitable, rather than a decentralized option that blockchain is

2. **Customization Needed**:
   If transactions between two or more parties have to be highly customized and are constantly changing, a blockchain solution would not be advisable due to its static design in nature

3. **Fast Performance Required**:
   One of the key challenges of blockchain technology is it is slow in terms of transactions per second. The scaling challenge has been mentioned over and over.
   Existing relational database solutions have proven themselves to be capable of running trillions of queries in a very short timeframe. Thus, sticking to the traditional DBs are advisable for the mean time until the tech catches up.

> Applications which Blockchain is suitable for?

SMART CONTRACTS USE CASES

Smart contracts are like regular contracts except the rules of the contract are enforced in real-time on a blockchain, which eliminates the middleman and adds levels of accountability for all parties involved in a way not possible with traditional agreements. This saves businesses time and money, while also ensuring compliance from everyone involved.

Blockchain-based contracts are becoming more and more popular as sectors like government, healthcare and the real estate industry discover the benefits. Below are a few examples of how companies are using blockchain to make contracts smarter.

MONEY TRANSFER USE CASES

Pioneered by Bitcoin, cryptocurrency transfer apps are exploding in popularity right now. Blockchain is especially popular in finance for the money and time it can save financial companies of all sizes.

By eliminating bureaucratic red tape, making ledger systems real-time and reducing third-party fees, blockchain can save the largest banks $8-$12 billion a year, according to a recent article by ComputerWorld. We'll take a deeper dive into four companies using blockchain to efficiently transfer money.

INTERNET OF THINGS USE CASES

The Internet of Things (IoT) is the next logical boom in blockchain applications. IoT has millions of applications and many safety concerns, and an increase in IoT products means better chances for hackers to steal your data on everything from an Amazon Alexa to a smart thermostat.

Blockchain-infused IoT adds a higher level of security to prevent data breaches by utilizing transparency and virtual incorruptibility of the technology to keep things "smart." Below are a few US companies using blockchain to make the Internet of Things safer and smarter.

> Applications which Blockchain is suitable for?

PERSONAL IDENTITY SECURITY USE CASES

According to identity theft expert LifeLock, more than 16 million Americans complained of identity fraud and theft in 2017 alone, with an identity being stolen every two seconds. Fraud on this scale can occur via everything from forged documents to hacking into personal files.

By keeping social security numbers, birth certificates, birth dates and other sensitive information on a decentralized blockchain ledger, the government could see a drastic drop in identity theft claims. Here are a few blockchain-based enterprises at the forefront of identity security.

LOGISTICS USE CASES

A major complaint in the shipping industry is the lack of communication and transparency due to the large number of logistics companies crowding the space. According to a joint study by Accenture and logistics giant DHL, there are more than 500,000 shipping companies in the US alone, causing data siloing and transparency issues. The report goes on to say blockchain can solve many of the problems plaguing logistics and supply chain management.

The groundbreaking study argues that blockchain enables data transparency by revealing a single source of truth. By acknowledging data sources, blockchain can build greater trust within the industry. The technology can also make the logistics process leaner and more automated, potentially saving the industry billions of dollars a year. Blockchain is not only safe, but a cost-effective solution for the logistics industry. Here are some companies on the cutting-edge of logistics blockchain technology.

GOVERNMENT USE CASES

One of the most surprising applications for blockchain can be in the form of improving government. As mentioned previously, some state governments like Illinois are already using the technology to secure government documents, but blockchain can also improve bureaucratic efficiency, accountability and reduce massive financial burdens. Blockchain has the potential to cut through millions of hours of red tape every year, hold public officials accountable through smart contracts and provide transparency by recording a public record of all activity, according to the New York Times.

Blockchain could also revolutionize our elections. Currently, voter apathy in the US is at an all-time high, with just over 58 percent turning out in the 2016 presidential election, while only 36.4 percent of the voting-eligible public showed up for \the 2014 midterm elections, according to PBS. Blockchain-based voting could improve civic engagement by providing a level of security and incorruptibility that allows voting to be done on mobile devices.

MEDIA USE CASES

Many of the current problems in media deal with data privacy, royalty payments and piracy of intellectual property. According to a study by Deloitte, the digitization of media has caused widespread sharing of content that infringes on copyrights. Deloitte believes blockchain can give the industry a much needed facelift when it comes to data rights, piracy and payments.

Blockchain's strength in the media industry is its ability to prevent a digital asset, such as an mp3 file, from existing in multiple places. It can be shared and distributed while also preserving ownership, making piracy virtually impossible through a transparent ledger system. Additionally, blockchain can maintain data integrity, allowing advertising agencies to target the right customers, and musicians to receive proper royalties for original works. The following US-based companies are helping grow the popularity of blockchain in our media.

#database-system! Blockchain Moment

26.8 Emerging Applications

Having seen how blockchains work and the benefits they offer, we can look at areas where blockchain technology is currently in use or may be used in the near future.

Applications most likely to benefit from the use of a blockchain are those that have high-value data, including possibly historical data, that need to be kept safe from malicious modification. Updates would consist mostly of appends in such applications. Another class of applications that are likely to benefit area those that involve multiple cooperating parties, who trust each other to some extent, but not fully, and desire to have a shared record of transactions that are digitally signed, and are kept safe from tampering. In this latter case, the cooperating parties could include the general public.

Below, we provide a list of several application domains along with a short explanation of the value provided by a blockchain implementation of the application. In some cases, the value added by a blockchain is a novel capability; in others, the value added is the ability to do something that could have been done previously only at prohibitive cost.

- **Academic certificates and transcripts:** Universities can put student certificates and transcripts on a public blockchain secured by the student's public key and signed digitally by the university. Only the student can read the records, but the student can then authorize access to those records. As a result, students can obtain certificates and transcripts for future study or for prospective employers in a secure manner from a public source. This approach was prototyped by MIT in 2017.
- **Accounting and audit:** Double-entry bookkeeping is a fundamental principle of accounting that helps ensure accurate and auditable records. A similar benefit can be gained from cryptographically signed blockchain entries in a digital distributed ledger. In particular, the use of a blockchain ensures that the ledger is tamperproof, even against insider attacks and hackers who may gain control of the database. Also, if the enterprise's auditor is a participant, then ledger entries can become visible immediately to auditors, enabling a continuous rather than periodic audit.
- **Asset management:** Tracking ownership records on a blockchain enables verifiable access to ownership records and secure, signed updates. As an example, real-estate ownership records, a matter of public record, could be made accessible to the public on a blockchain, while updates to those records could be made only by transactions signed by the parties to the transaction. A similar approach can be applied to ownership of financial assets such as stocks and bonds. While stock exchanges manage trading of stocks and bonds, long term records are kept by depositories that users must trust. Blockchain can help track such assets without having to trust a depository.
- **E-government:** A single government blockchain would eliminate agency duplication of records and create a common, authoritative information source. A highly notable user of this approach is the government of Estonia, which uses its blockchain for taxation, voting, health, and an innovative "e-Residency" program.

#database-system! Blockchain Moment

- **Foreign-currency exchange:** International financial transactions are often slow and costly. Use of an intermediary cryptocurrency can enable blockchain-based foreign-currency exchange at a relative rapid pace with full, irrefutable traceability. Ripple is offering such capability using the XRP currency.
- **Health care:** Health records are notorious for their nonavailability across health-care providers, their inconsistency, and their inaccuracy even with the increased use of electronic health records. Data are added from a large number of sources and the provenance of materials used may not be well documented (see discussion of supply chains below). A unified blockchain is suitable for distributed update,

  and cryptographic data protection, unlockable by the patient's private key, would enable access to a patient's full health record anytime, anywhere in an emergency. The actual records may be kept offchain, but the blockchain acts as the trusted mechanism for accessing the data.
- **Insurance claims:** The processing of insurance claims is a complex workflow of data from the scene of the claim, various contractors involved in repairs, statements from witnesses, etc. A blockchain's ability to capture data from many sources and distribute it rapidly, accurately, and securely, promises efficiency and accuracy gains in the insurance industry.
- **Internet of things:** The Internet of Things (IoT) is a term that refers to systems of many interacting devices ("things"), including within smart buildings, smart cities, self-monitoring civil infrastructure, and so on. These devices could act as nodes that can pass blockchain transactions into the network without having to ensure the transmission of data reaches a central server. In the late 2010s, research is underway to see if this data-collection approach can be effective in lowering costs and increasing performance. Adding so many entries to a blockchain in a short period of time may suggest a replacement of the chain data structure with a directed, acyclic graph. The Iota blockchain is an example of one such system, where the graph structure is called a *tangle*.
- **Loyalty programs and aggregation of transactions:** There are a variety of situations where a customer or user makes multiple purchases from the same vendor, such as within a theme park, inside a video game, or from a large online retailer. These vendors could create internal cryptocurrencies in a proprietary, permissioned blockchain, with currency value pegged to a fiat currency like the dollar. The vendor gains by replacing credit-card transactions with vendor-internal transactions. This saves credit-card fees and allows the vendor to capture more of the valuable customer data coming from these transactions. The same concept can apply to retail loyalty points, exemplified by airline frequent-flyer miles. It is costly for vendors to maintain these systems and coordinate with partner vendors in the program. A blockchain-based system allows the hosting vendor to distribute the workload among the partners and allows transactions to be posted in a decentralized manner, releaving the vendor from have to run its own online transaction processing system. In the late 2010s, business strategies were being tested around these concepts.

#database-system! Blockchain Moment

- **Supply chain:** Blockchain enables every participant in a supply chain to log every action. This facilitates tracking the movement of every item in the chain rather than only aggregates like crates, shipments, etc. In the event of a recall, the set of affected products can be pinpointed to a smaller set of products and done so quickly. When a quality issue suggests a recall, some supply-chain members may be tempted to cover up their role, but the immutability of the blockchain prevents record falsification after the fact.

- **Tickets for events:** Suppose a person *A* has bought tickets for an event, but now wishes to sell them, and *B* buys the ticket from *A*. Given that tickets are all sold online, *B* would need to trust that the ticket given by *A* is genuine, and *A* has not already sold the ticket, that is, the ticket has not been double-spent. If ticket transactions are carried out on a blockchain, double-spending can be detected easily. Tickets can be verified if they are signed digitally by the event organizer (whether or not they are on a blockchain).
- **Trade finance:** Companies often depend on loans from banks, issued through letters of credits, to finance purchases. Such letters of credit are issued against goods based on bills of lading indicating that the goods are ready for shipment. The ownership of the goods (title) is then transferred to the buyer. These transactions involve multiple parties including the seller, buyer, the buyer's bank, the seller's bank, a shipping company and so forth, which trust each other to some extent, but not fully. Traditionally, these processes were based on physical documents that have to be signed and shipped between parties that may be anywhere on the globe, resulting in significant delays in these processes. Blockchain technology can be used to keep these documents in a digital form, and automate these processes in a way that is highly secure yet very fast (at least compared to processing of physical documents).

Other applications beyond those we have listed continue to emerge.

#database-system! Blockchain Moment

26.9 Summary

- Blockchains provide a degree of privacy, anonymity, and decentralization that is hard to achieve with a traditional database.
- Public blockchains are accessibly openly on the internet. Permissioned blockchains are managed by an organization and usually serve a specific enterprise or group of enterprises.
- The main consensus mechanisms for public blockchains are proof-of-work and proof-of-stake. Miners compete to add the next block to the blockchain in exchange for a reward of blockchain currency.
- Many permissioned blockchains use a Byzantine consensus algorithm to choose the node to add the next block to the chain.
- Nodes adding a block to the chain first validate the block. Then all full nodes maintaining a replica of the chain validate the new block.
- Key blockchain properties include irrefutability and tamper resistance.
- Cryptographic hash functions must exhibit collision resistance, irreversibility, and puzzle friendliness.
- Public-key encryption is based on a user having both a public and private key to enable both the encryption of data and the digital signature of documents.
- Proof-of-work requires a large amount of computation to guess a successful nonce that allows the hash target to be met. Proof-of-stake is based on ownership of blockchain currency. Hybrid schemes are possible.
- Smart contracts are executable pieces of code in a blockchain. In some chains, they may operate as independent entities with their own data and account. Smart contracts may encode complex business agreements and they may provide ongoing services to nodes participating in the blockchain.
- Smart contracts get input from the outside world via trusted oracles that serve as a real-time data source.
- Blockchains that retain state can serve in a manner similar to a database system and may benefit from the use of database indexing methods and access optimization, but the blockchain structure may place limits on this.